

Claude CCAF - Mock Exams

All three mock papers with answers, full explanations and exam tips.

171 questions across 3 chapters

Generated 2026-08-24

Claude CCAF Practice Mock 1 (Foundations & Architecture - 60Q)

Mock Tests - 58 questions

1. An MCP server exposes a `query_database` tool for Snowflake. Agents ignore it and run SQL via the built-in Bash tool against the Snowflake CLI, even though the MCP tool returns structured results with column types and pagination that Bash does not. What is the most likely cause and fix?
- (A) The MCP server is not properly connected. Restart the MCP server and verify the connection with a test query.
 - (B) Enhance the sparse description to spell out the tool's structured output and pagination advantages over Bash.
 - (C) Disable the Bash tool entirely so the agent is forced to use the MCP tool for all operations.
 - (D) Add a system prompt instruction telling the agent to always use `query_database` instead of Bash for SQL queries.

Answer: (B) Enhance the sparse description to spell out the tool's structured output and pagination advantages over Bash.

EXPLANATION

When MCP tool descriptions are sparse, agents default to the familiar Bash tool. Enhancing the description to explain the structured results, column types, and pagination gives the agent enough to prefer the MCP tool for database queries.

Why the other options are wrong

- **A.** If the server were disconnected, the tool would not appear in the available tools list at all. The agent is choosing not to use it, which is a description problem, not a connectivity problem.
- **C.** Disabling Bash removes a critical general-purpose tool needed for many operations beyond database queries. The fix should make the MCP tool more attractive for its specific use case, not remove other tools.
- **D.** Whilst a system prompt instruction could work as a short-term fix, it is brittle and does not scale. The root cause is the sparse description. Enhancing the description is the sustainable fix that works across all agents and contexts.

EXAM TIP

Lesson 2.4: MCP Server Integration (Enhancing MCP descriptions)

2. A multi-agent research system must process a customer's request that involves three sequential stages: data collection, analysis, and report generation. Each stage depends on the output of the previous one. Which orchestration pattern is most appropriate?
- (A) Parallel orchestration - run all three subagents simultaneously to minimise latency
 - (B) Pipeline orchestration - pass the output of each stage as input to the next in a defined sequence
 - (C) Dynamic adaptive decomposition - let the coordinator decide the order at runtime based on query complexity
 - (D) Hub-and-spoke with all three agents reporting independently to the coordinator

Answer: (B) Pipeline orchestration - pass the output of each stage as input to the next in a defined sequence

EXPLANATION

Pipeline orchestration is the correct pattern for sequential dependencies. Each stage completes before the next begins, with the output of one stage serving as the input to the next. This matches the data collection -> analysis -> report generation dependency chain.

Why the other options are wrong

- **A.** Parallel execution requires independent tasks. Here, analysis depends on data collection results and report generation depends on analysis results. Running them simultaneously would mean the analysis agent has no data to analyse.
- **C.** Dynamic adaptive decomposition is for open-ended investigation tasks where the next step depends on discoveries. Here, the three stages and their dependencies are known in advance, making a fixed pipeline the more predictable and efficient choice.
- **D.** Independent reporting ignores the sequential dependencies. The analysis agent cannot produce meaningful results without data collection output. The pipeline must enforce the execution order.

EXAM TIP

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

3. The moderation system's classification schema has a 'category' field defined as a free-text string. Auditors find 47 different category values in production data, including 'hate speech', 'Hate Speech', 'hate-speech', 'hateful content', and 'hate_speech' - all intended to be the same category. This makes downstream analytics and routing unreliable. What is the best schema fix?
- (A) Add a post-processing normalisation step that maps all variations to canonical category names
 - (B) Change 'category' from free-text to an enum with values like 'hate_speech', 'spam', and 'harassment', plus an 'other' option.
 - (C) Add detailed instructions to the prompt listing the exact category names and their capitalisation so the model always emits the canonical string.
 - (D) Add few-shot examples showing the correct category formatting for each type

Answer: (B) Change 'category' from free-text to an enum with values like 'hate_speech', 'spam', and 'harassment', plus an 'other' option.

EXPLANATION

Enum fields constrain the model to predefined values, eliminating spelling and formatting variations. Including 'other' as an enum value handles edge cases without allowing free-text drift. With strict: true, the schema guarantees only valid enum values are returned.

Why the other options are wrong

- **A.** Post-processing normalisation adds complexity and requires constant maintenance as new variations appear. The schema should prevent the problem at the source rather than patching it downstream.
- **C.** Prompt instructions are probabilistic. The model may still produce variations despite instructions. Schema-level enforcement via enums is deterministic and cannot be overridden.
- **D.** Few-shot examples improve consistency but cannot guarantee exact string matching. Schema enums are the correct mechanism for constraining categorical output to a fixed set of values.

EXAM TIP

Lesson 4.2: Structured Output with Tool Use (Enum schema)

4. A research agent calls an external API via an MCP server. After the 30th query in a batch of 50, the API starts returning HTTP 429 errors. The MCP server returns a generic 'Request failed' for every failure, so the agent abandons the batch after three consecutive failures. What MCP server change would most improve resilience?
- (A) Implement automatic retry with exponential backoff inside the MCP server, hiding rate limits from the agent entirely.
 - (B) Return `errorCategory: 'transient', isRetryable: true`, with a `retryAfterMs` field telling the agent how long to wait.
 - (C) Return `errorCategory: 'business', isRetryable: false` to tell the agent to stop making requests entirely.
 - (D) Queue all 50 requests at the MCP server level and process them sequentially with built-in rate limiting.

Answer: (B) Return `errorCategory: 'transient', isRetryable: true`, with a `retryAfterMs` field telling the agent how long to wait.

EXPLANATION

Rate limiting is a transient error - it will resolve after a delay. Structured metadata with the specific retry delay lets the agent space out remaining queries intelligently rather than abandoning the batch. The agent can continue processing other tasks while waiting.

Why the other options are wrong

- **A.** Hiding rate limits from the agent removes its ability to make informed decisions, such as prioritising remaining queries, switching to cached results, or reporting partial progress. The agent should receive structured metadata to decide its own recovery strategy.
- **C.** Rate limits are transient (they resolve after a delay), not business errors (which represent policy or rule violations the agent cannot recover from). Marking them as non-retryable causes the agent to abandon 20 remaining queries that would succeed after a brief delay.
- **D.** Server-side queuing removes the agent's ability to prioritise, cancel, or reorder queries based on intermediate results. It also creates a long-running blocking call that prevents the agent from doing other work.

EXAM TIP

Lesson 2.2: Structured Error Responses (Transient errors and retry-after)

5. Your manager proposes switching both your blocking pre-merge code review and your overnight technical debt report to the Message Batches API for 50% cost savings. How should you evaluate this proposal?
- (A) Switch both to batch processing with status polling to check for completion
 - (B) Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks
 - (C) Keep real-time calls for both to avoid batch result ordering issues
 - (D) Switch both to batch with a timeout fallback to real-time if batches take too long

Answer: (B) Use batch processing for the technical debt reports only; keep real-time calls for pre-merge checks

EXPLANATION

Pre-merge checks are blocking workflows where developers wait. The batch API's 24-hour window makes it unsuitable. Technical debt reports are overnight and latency-tolerant.

Why the other options are wrong

- **A.** Batch processing has no guaranteed latency SLA. Polling does not make it suitable for blocking pre-merge checks.
- **C.** Batch results can be correlated using custom_id fields. Ordering is not the issue - latency requirements are.
- **D.** This adds unnecessary complexity. Match each API to its latency requirements: synchronous for blocking, batch for tolerant.

EXAM TIP

Lesson 4.5: Batch Processing and Prompt Optimisation (Batches API facts, SLA implications); Anthropic: Message Batches API

6. A web search subagent in a multi-agent research system times out while researching a complex topic. You need to design how this failure information flows back to the coordinator. Which approach best enables intelligent recovery?

- (A) Return structured error context including failure type, attempted query, partial results, and potential alternative approaches
- (B) Implement automatic retry with exponential backoff, returning a generic 'search unavailable' status only after all retries are exhausted
- (C) Catch the timeout and return an empty result set marked as successful
- (D) Propagate the timeout exception to a top-level handler that terminates the entire research workflow

Answer: (A) Return structured error context including failure type, attempted query, partial results, and potential alternative approaches

EXPLANATION

This gives the coordinator everything it needs to decide: retry with modified query, try an alternative approach, or proceed with partial results.

Why the other options are wrong

- **B.** The generic status hides valuable context from the coordinator, preventing informed recovery decisions.
- **C.** Silent suppression prevents any recovery. The coordinator believes the search succeeded and found nothing, so it will not attempt alternatives.
- **D.** Workflow termination wastes partial results from other subagents that may have completed successfully.

EXAM TIP

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context)

7. The moderation system's agentic loop uses a hardcoded decision tree: if the classify_content tool returns 'hate_speech', always call escalate_to_human; if it returns 'spam', always call auto_remove. During testing, the team discovers that satirical posts criticising hate speech are being auto-escalated, and sophisticated spam disguised as legitimate marketing slips through. What is the architectural problem?

- (A) The classify_content tool needs more granular category labels so it can tell satire criticising hate speech apart from genuine hate speech.
- (B) Replace the hardcoded decision tree with model-driven decisions, letting Claude weigh the full context of each post before it acts.
- (C) Add a confidence threshold so only high-confidence classifications trigger automatic actions
- (D) Route all ambiguous cases to human review to avoid misclassification

Answer: (B) Replace the hardcoded decision tree with model-driven decisions, letting Claude weigh the full context of each post before it acts.

EXPLANATION

Hardcoded decision trees treat classification labels as absolute when real content is nuanced. Letting Claude reason about context (satire vs genuine hate, sophisticated spam patterns) produces better moderation decisions. The agentic loop should let the model decide, not map labels to fixed actions.

Why the other options are wrong

- **A.** More granular labels do not address the core issue. A hardcoded decision tree cannot adapt to nuance, such as satire versus genuine hate, no matter how fine the labels are. The fix is model-driven reasoning.
- **C.** Confidence thresholds reduce false positives but the fundamental problem remains: a hardcoded tree cannot handle contextual nuance. Low-confidence cases still need model-driven reasoning, not just deferral.
- **D.** Routing everything ambiguous to humans defeats the purpose of automated moderation. The model can reason about context effectively when given the opportunity; the hardcoded tree is what prevents it.

EXAM TIP

Lesson 1.1: Agentic Loops (Model-driven decision-making, Anti-patterns)

8. A documentation team needs to simultaneously update API reference docs for three independent microservices after a breaking change. Each update requires reading source code, updating Markdown files, and validating links. A single Claude Code session would exhaust the context window trying to hold all three services' code simultaneously. What is the recommended approach?

(A) Process the three services sequentially in the same session, running `/compact` between each service to free context
(B) Use git worktree for three branches, each with its own Claude Code session on one service, then merge the results
(C) Create a single skill with context: fork that processes all three services in parallel within one session
(D) Split the documentation files into smaller chunks and process each chunk in a separate API call using the batch API

Answer: (B) Use git worktree for three branches, each with its own Claude Code session on one service, then merge the results

EXPLANATION

git worktree creates separate working directories on different branches, each with its own isolated Claude Code session. Each session has a full context budget dedicated to one service. The three updates run in parallel without interfering with each other, and results are merged via standard git workflows.

Why the other options are wrong

- **A.** Even with `/compact`, each service's context (source code, documentation, edits) is substantial. Sequential processing risks context degradation as compaction summaries lose detail from earlier services, and it is slower than parallel processing.
- **C.** context: fork isolates a skill's output from the main conversation but does not create multiple parallel execution contexts. A single skill cannot simultaneously process three independent services in parallel.
- **D.** The batch API is for high-throughput, latency-tolerant workloads - not for interactive Claude Code sessions. Documentation updates require interactive exploration of source code and iterative editing, which the batch API does not support.

EXAM TIP

Anthropic: Claude Code Documentation; Git: Worktrees

9. The data platform team has built an MCP server with 22 tools: `query_snowflake`, `query_postgres`, `query_api`, plus 19 specialised tools for individual data transformations (`pivot_table`, `calculate_percentile`, `normalise_currency`, etc.). Agents take 3-4 turns to select the correct tool and frequently choose the wrong transformation. What is the most effective redesign?

(A) Improve all 22 tool descriptions with detailed examples and boundary conditions to help the agent distinguish between them.
(B) Consolidate the 19 transformation tools into a single `transform_data` tool with a `transform_type` parameter, reducing the total to 4 tools.
(C) Use `tool_choice: 'any'` to force the agent to always call a tool, eliminating turns where the agent reasons without acting.
(D) Split the tools across two separate MCP servers - one for queries and one for transformations - to reduce cognitive load.

Answer: (B) Consolidate the 19 transformation tools into a single `transform_data` tool with a `transform_type` parameter, reducing the total to 4 tools.

EXPLANATION

Reducing from 22 to 4 tools brings the count within the optimal 4-5 range for reliable tool selection. The 19 transformation tools share a common pattern (input data, transformation type, output format) and are natural candidates for consolidation into a single parameterised tool.

Why the other options are wrong

- **A.** Better descriptions help, but 22 tools still exceeds the practical limit for reliable selection. Research shows tool selection degrades significantly beyond 4-5 tools per agent. The tool count itself is the problem.
- **C.** Forcing tool calls does not improve selection accuracy - it just ensures the agent picks something, potentially the wrong tool. The root cause is too many similar tools, not too few tool calls.
- **D.** MCP server boundaries are invisible to the agent. All tools from all connected servers appear in a single list. Splitting across servers does not reduce the number of tools the agent must choose from.

EXAM TIP

Lesson 2.3: Tool Distribution and Tool Choice (Consolidating near-duplicate tools, Tool overload)

10. A documentation team creates a /generate-api-docs skill that reads source code files and produces Markdown API reference pages. The skill generates verbose output (200+ lines per endpoint) and should be available to every team member who clones the repository. How should the skill be configured?

- (A) Create a SKILL.md in ~/.claude/skills/ with context: fork frontmatter, and instruct each team member to copy it locally
- (B) Create a SKILL.md in .claude/skills/ with context: fork, so it is shared via git and isolates the verbose output
- (C) Add the documentation generation instructions to the root CLAUDE.md so they load automatically in every session
- (D) Create a SKILL.md in .claude/skills/ without any frontmatter, relying on the team to manually manage context overflow

Answer: (B) Create a SKILL.md in .claude/skills/ with context: fork, so it is shared via git and isolates the verbose output

EXPLANATION

.claude/skills/ is project-scoped and version-controlled, making it available to every developer who clones the repository. context: fork isolates the verbose 200+ line output per endpoint into a forked context, preventing it from consuming the main conversation's context budget.

Why the other options are wrong

- **A.** ~/.claude/skills/ is user-scoped and not version-controlled. Requiring manual copying defeats the purpose of team-shared configuration and creates maintenance drift.
- **C.** CLAUDE.md is for always-loaded standards, not for on-demand command definitions. Loading verbose generation instructions in every session wastes context tokens when the team is doing non-documentation work.
- **D.** Without context: fork, the verbose output (200+ lines per endpoint across many endpoints) would flood the main conversation context. The context: fork frontmatter is essential for isolating bulky output.

EXAM TIP

Lesson 3.2: Custom Slash Commands and Skills (context: fork); Claude Code: Skills

11. Claude Code is synthesising release notes from commit messages, PR descriptions, and changelog entries. During a 200+ commit session, summaries of early commits become vague ('various bug fixes') while recent commits are still detailed accurately. The context window is not full. What is the most likely cause?

- (A) The model's temperature is set too high, causing it to generate vague summaries randomly
- (B) The lost-in-the-middle effect: the model favours the start and end of long input, losing middle commit detail.
- (C) The commit messages for early commits are inherently less detailed than recent ones, so the vague summaries are accurate
- (D) Claude Code applies progressive summarisation to older commits to conserve context for recent ones

Answer: (B) The lost-in-the-middle effect: the model favours the start and end of long input, losing middle commit detail.

EXPLANATION

The lost-in-the-middle effect is a well-documented phenomenon where models attend more strongly to the start and end of long contexts, with reduced attention to middle content. With 200+ commits loaded sequentially, commits in the middle of the sequence receive less attention, producing vague summaries. Processing in smaller batches or reordering critical information to the start and end mitigates this.

Why the other options are wrong

- **A.** Temperature affects output randomness but would produce inconsistent quality across all commits, not a systematic pattern where early commits are vague and recent commits are accurate.
- **C.** The question states the vagueness applies to early commits specifically, not commits with poor messages. The systematic pattern (early = vague, recent = detailed) points to a context positioning effect, not source quality variation.
- **D.** Claude Code does not automatically apply progressive summarisation to tool results within a single processing step. The context window is not full, so there is no space pressure triggering summarisation. The issue is attention distribution across long inputs.

EXAM TIP

Lesson 5.1: Context Window Management (Lost in the middle)

12. A research team is using Claude Code to analyse a large dataset. After completing an initial analysis, they want to explore two competing hypotheses: one using a statistical modelling approach and another using a machine learning approach. Both explorations should start from the same baseline analysis but proceed independently. Which session management strategy is correct?

- (A) Resume the session twice with `--resume`, once for each hypothesis, running them one after the other
- (B) Start two fresh sessions, each with an injected summary of the initial analysis, and explore one hypothesis in each
- (C) Use the initial session and explore both hypotheses sequentially, asking the agent to set aside the first approach before starting the second
- (D) Use `fork_session` to create two independent branches from the shared analysis baseline, exploring one hypothesis in each fork

Answer: (D) Use `fork_session` to create two independent branches from the shared analysis baseline, exploring one hypothesis in each fork

EXPLANATION

`fork_session` creates independent branches from a shared baseline, which is exactly the use case here. Both forks start from the complete initial analysis but proceed independently. Neither exploration contaminates the other, enabling a clean comparison of the two approaches.

Why the other options are wrong

- **A.** Resuming the same session twice would mean the second resume inherits context from the first hypothesis exploration, contaminating the independent baseline. Each hypothesis needs to diverge from the original analysis, not from each other.
- **B.** While this avoids context contamination, it loses the full richness of the original analysis context. Fresh start with summary injection is best when tool results are stale. Here, the baseline analysis is still valid and both branches should start from the complete shared baseline.
- **C.** Sequential exploration in the same session means the second hypothesis is influenced by the first - the agent carries context, conclusions, and biases from the first exploration. Independent exploration requires isolated branches.

EXAM TIP

Lesson 1.7: Session State and Resumption (Session management options); Lesson 1.3: Subagent Invocation and Context Passing (`fork_session`); Anthropic: Claude Code Documentation

13. A documentation generation pipeline uses Claude Code to produce docs from three sources: source code comments, existing wiki pages, and API schema files. During a run, the wiki page retrieval fails with a timeout, but the source code and API schema are available. The pipeline currently halts entirely on any source failure. What is the best error handling strategy?

- (A) Retry the wiki retrieval three times with exponential backoff. If all retries fail, halt the pipeline to prevent incomplete documentation
- (B) Return structured error context for the wiki failure, proceed with available sources, and mark the gaps that lack wiki content.
- (C) Silently skip the wiki source and generate documentation from the remaining two sources without noting the omission
- (D) Use the source code comments to infer what the wiki pages would have contained, filling in the gaps with generated content

Answer: (B) Return structured error context for the wiki failure, proceed with available sources, and mark the gaps that lack wiki content.

EXPLANATION

This produces maximum value from available sources while maintaining transparency about what is missing. Structured error context enables intelligent recovery (retry later, alert the team). Explicit gap markers prevent users from trusting incomplete documentation as complete.

Why the other options are wrong

- **A.** Halting the entire pipeline because one of three sources is unavailable wastes the successful results from the other two sources. Documentation from source code and API schemas is still valuable even without wiki content.
- **C.** Silent omission hides the failure. Users would trust the documentation as complete, not knowing that wiki-sourced context (which may contain critical operational notes or caveats) is missing.
- **D.** Inferring wiki content from source code produces hallucinated documentation that appears authoritative. Wiki pages often contain operational context, known issues, and tribal knowledge that cannot be inferred from code.

EXAM TIP

Lesson 5.3: Error Propagation in Multi-Agent Systems (Structured error context, Coverage annotations)

14. A search_papers MCP tool has three failure patterns: (1) the upstream academic API returns HTTP 503 during peak hours, (2) users request papers from a restricted journal the system has no licence for, and (3) the agent submits a malformed DOI like 'doi-1234' that fails the input regex. The team wants structured error metadata so agents can handle each case differently. Which `errorCategory` and `isRetryable` combination is correct for all three?

(A) All three should be `errorCategory: 'transient', isRetryable: true`, since they all prevent the tool from completing its task.

(B) HTTP 503: `transient/retryable`; restricted journal: `business/not retryable`; malformed DOI: `validation/retryable`.

(C) HTTP 503: `transient/retryable`; restricted journal: `transient/retryable`; malformed DOI: `transient/retryable`.

(D) HTTP 503: `validation/retryable`; restricted journal: `business/not retryable`; malformed DOI: `business/not retryable`.

Answer: (B) HTTP 503: `transient/retryable`; restricted journal: `business/not retryable`; malformed DOI: `validation/retryable`.

EXPLANATION

HTTP 503 is a temporary upstream outage (transient, will likely resolve on retry). Restricted journal is a policy limitation (business, will never resolve on retry - escalate or offer an alternative source). Malformed DOI is validation; the agent must repair the input and then retry.

Why the other options are wrong

- **A.** Lumping every failure into 'transient' is the canonical anti-pattern. The restricted journal case is a business error (licence policy); the agent must escalate, not retry. The malformed DOI is validation; the agent must repair the input format first.

- **C.** The restricted journal error is not transient - the system lacks a licence, and no amount of retrying will grant access. A malformed DOI is validation, not transient; the agent must repair the input rather than blindly retry the same value.

- **D.** HTTP 503 is transient (a temporary upstream outage), not validation - nothing is wrong with the request input. A malformed DOI is validation (fix input and retry), not business - business errors represent policy or rule violations, not input format problems.

EXAM TIP

Lesson 2.2: Structured Error Responses (Four error categories)

15. The moderation system uses `tool_use` with a JSON schema for classification output. All fields including 'sub_category' and 'target_demographic' are marked as required. Auditors discover that when a post is spam (which has no target demographic), the model fabricates plausible-sounding demographics like 'general public' or 'young adults.' What schema change prevents this fabrication?

(A) Add a validation step that rejects `target_demographic` values for spam posts

(B) Make 'target_demographic' nullable so the model can return null when the field does not apply instead of fabricating a value.

(C) Remove 'target_demographic' from the schema entirely since it causes fabrication

(D) Add a prompt instruction telling the model to leave 'target_demographic' empty whenever a post is spam and the field does not apply.

Answer: (B) Make 'target_demographic' nullable so the model can return null when the field does not apply instead of fabricating a value.

EXPLANATION

Required fields pressure the model to produce a value even when none exists. Making the field nullable gives the model a legitimate way to indicate 'not applicable,' eliminating the incentive to fabricate. This is the standard pattern for fields that apply conditionally.

Why the other options are wrong

- **A.** Post-hoc validation catches the problem but does not prevent it. The model still fabricates values; they are just rejected after the fact. Schema design should prevent fabrication at the source.

- **C.** Removing the field loses valuable data for categories where target demographic is genuinely relevant (hate speech, harassment). The fix is making it nullable, not removing it.

- **D.** Prompt instructions cannot override a schema that marks the field as required. The JSON schema enforces the constraint at the API level, and the model must comply with the schema regardless of prompt instructions.

EXAM TIP

Lesson 4.2: Structured Output with Tool Use (Nullable fields)

16. A multi-agent research system has three subagents (financial filings, news, technical white papers) and a synthesis agent. Each subagent returns properly attributed findings, but the final synthesised report has no source attribution: stakeholders cannot trace which claim came from which source. Which fix addresses the root cause?

- (A) Append a bibliography section at the end of the report listing all sources each subagent consulted
- (B) Have each subagent include source URLs as inline hyperlinks in their prose output so the synthesis agent can preserve them
- (C) Require subagents to output structured claim-source mappings and instruct the synthesis agent to preserve and merge them.
- (D) Store all subagent outputs in a database and have the synthesis agent reference database entries by ID instead of incorporating content directly

Answer: (C) Require subagents to output structured claim-source mappings and instruct the synthesis agent to preserve and merge them.

EXPLANATION

Structured claim-source mappings survive synthesis because they are data structures, not prose that gets rewritten. The synthesis agent can merge mappings from multiple subagents while preserving the link between each claim and its source. This also enables content-appropriate rendering: financial data as tables, news as prose, technical findings as lists - each with attribution intact.

Why the other options are wrong

- **A.** A bibliography lists sources but does not map specific claims to specific sources. Stakeholders need to know which claim came from which document, not just which documents were consulted overall. This is document-level attribution, not claim-level provenance.
- **B.** Inline hyperlinks in prose are fragile - the synthesis agent will rewrite, merge, and compress prose during summarisation, stripping or disconnecting links from their associated claims. Attribution must be in structured data, not embedded in prose that gets rewritten.
- **D.** This adds infrastructure complexity without solving the synthesis problem. The synthesis agent still needs to merge and present findings coherently. Database references do not prevent attribution loss during the summarisation and rewriting that synthesis inherently requires.

EXAM TIP

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings, Attribution preservation)

17. The data platform's query_snowflake tool has a description that reads: 'Queries Snowflake data warehouse. Accepts SQL.' The agent correctly uses the tool but frequently sends queries using PostgreSQL-specific syntax (e.g. string_agg, which Snowflake spells LISTAGG) that Snowflake rejects. What is the most effective fix?

- (A) Add a SQL syntax validation layer in front of the MCP tool that rejects non-Snowflake syntax before execution.
- (B) State in the tool description that it expects the Snowflake SQL dialect, not PostgreSQL.
- (C) Add a system prompt instruction listing all Snowflake-specific SQL functions the agent should use.
- (D) Have the MCP server automatically translate PostgreSQL syntax to Snowflake syntax before executing the query.

Answer: (B) State in the tool description that it expects the Snowflake SQL dialect, not PostgreSQL.

EXPLANATION

The description says 'Accepts SQL' without naming the dialect, so the agent defaults to PostgreSQL syntax. Naming the Snowflake dialect and giving examples in the description (LISTAGG rather than string_agg, ILIKE is supported) resolves the misrouting at source.

Why the other options are wrong

- **A.** A validation layer adds infrastructure complexity and maintenance burden. The root cause is that the tool description does not specify which SQL dialect to use. Fixing the description is the proportionate first step.
- **C.** A system prompt instruction is brittle and consumes context tokens for every turn, even when the Snowflake tool is not being used. The tool description is the correct place for tool-specific dialect guidance.
- **D.** Auto-translation is fragile and cannot handle all dialect differences. It masks the problem rather than fixing it and introduces a complex transformation layer prone to edge cases.

EXAM TIP

Lesson 2.1: Tool Interface Design (Dialect-specific descriptions)

18. Your Claude Code agent generates unit tests in the CI pipeline. When given detailed instructions alone, it produces tests with inconsistent assertion styles: sometimes using `expect().toBe()`, sometimes `assert.equal()`, and occasionally mixing both in the same file. Adding more detailed instructions about assertion style did not fix the problem. What should you do next?

- (A) Switch to a different model that better follows formatting instructions
- (B) Add a linter post-processing step to automatically convert all assertions to a single style
- (C) Add 2-4 few-shot examples showing complete test files with the desired assertion style and reasoning for why that style was chosen over alternatives
- (D) Add 2-4 few-shot examples demonstrating the desired assertion style with reasoning for each testing decision, covering edge cases like async functions and error handling

Answer: (D) Add 2-4 few-shot examples demonstrating the desired assertion style with reasoning for each testing decision, covering edge cases like async functions and error handling

EXPLANATION

Few-shot examples are the most effective technique when detailed instructions alone produce inconsistent formatting. Including 2-4 examples with reasoning teaches generalisation to novel patterns, not just pattern-matching. Covering varied scenarios (async, error handling) ensures the model generalises correctly.

Why the other options are wrong

- **A.** The issue is not model capability. When detailed instructions alone produce inconsistent formatting, the correct intervention is few-shot examples, not a model change.
- **B.** Post-processing masks the problem rather than solving it. The model should learn to produce consistent output directly, which few-shot examples achieve more effectively.
- **C.** This is partially correct but incomplete. The examples only cover the assertion style preference. Without covering varied scenarios (async functions, error handling), the model pattern-matches the specific cases shown rather than generalising the style consistently across all test types.

EXAM TIP

Lesson 4.2: Few-Shot Prompting (Few-shot examples); Anthropic: Multishot (Few-Shot) Prompting

19. The content moderation system uses a hub-and-spoke architecture with a coordinator that routes posts to specialist subagents: a text classifier, an image analyser, and a policy enforcer. The team notices that the image analyser sometimes directly calls the policy enforcer's action tools to remove posts, bypassing the coordinator. What is the architectural violation and how should it be fixed?

- (A) Give the image analyser its own copy of the policy enforcer's action tools so it can remove posts itself without calling another subagent.
- (B) The image analyser breaks hub-and-spoke isolation; scope its tools to image analysis and route all results back through the coordinator.
- (C) Merge the image analyser and policy enforcer into a single subagent to simplify the communication flow
- (D) Add a message queue between the image analyser and the policy enforcer so communication is asynchronous

Answer: (B) The image analyser breaks hub-and-spoke isolation; scope its tools to image analysis and route all results back through the coordinator.

EXPLANATION

In hub-and-spoke orchestration, subagents must be isolated and communicate only through the coordinator. The image analyser should return its classification result to the coordinator, which then decides whether to invoke the policy enforcer. Direct subagent-to-subagent communication breaks the architecture.

Why the other options are wrong

- **A.** Duplicating action tools across subagents creates inconsistent enforcement and makes policy changes harder to propagate. The issue is the communication pattern, not tool availability.
- **C.** Merging subagents with distinct responsibilities (analysis vs enforcement) violates separation of concerns. The image analyser and policy enforcer have fundamentally different roles and tool requirements.
- **D.** An async queue between subagents still bypasses the coordinator. In hub-and-spoke, all inter-agent communication must flow through the coordinator regardless of synchronicity.

EXAM TIP

Lesson 1.2: Multi-Agent Orchestration (Isolation principle, Hub-and-spoke architecture)

20. The moderation system validates that every classification includes a 'reasoning' field explaining the decision. When validation fails (empty reasoning), the system retries. For most posts, the retry succeeds after feeding back the error 'reasoning field was empty - provide a brief justification for the classification.' However, for posts in unfamiliar languages, retries consistently fail. What should the system do?

- (A) Increase the retry count from 1 to 5 for unfamiliar language posts since the model may succeed with more attempts
- (B) Retry format errors on analysable posts; route capability gaps like unfamiliar languages to human review.
- (C) Add the unfamiliar language to the system prompt as a supported language to encourage the model to try harder

(D) Remove the reasoning validation requirement for posts in unfamiliar languages

Answer: (B) Retry format errors on analysable posts; route capability gaps like unfamiliar languages to human review.

EXPLANATION

The retry boundary distinguishes between fixable format issues and unfixable capability gaps. Empty reasoning on analysable posts is a format error that retries with error feedback can fix. Unfamiliar languages represent absent capability - no amount of retrying will create language understanding. Route these to human review.

Why the other options are wrong

- **A.** More retries are ineffective when the model lacks the capability to analyse content in that language. The failure is not a transient format issue - the model genuinely cannot provide reasoning for content it cannot understand.
- **C.** Listing an unsupported language in the prompt does not give the model language understanding. This may actually worsen the problem by encouraging fabricated analysis of content the model cannot read.
- **D.** Removing the validation requirement means these posts would receive classifications without any justification, which undermines auditability. The correct approach is human review, not lower standards.

EXAM TIP

Lesson 4.4: Validation, Retry, and Feedback Loops (Retry boundary)

21. A developer's Claude Code applies the team's API conventions correctly in some sessions but not others, on the same project. They suspect the wrong memory files are loading in the failing sessions. What is the fastest way to confirm which CLAUDE.md and rules files a session has actually loaded?

- (A) Run `/memory` in the session to list the loaded memory files
- (B) Run `/compact` to reload the configuration hierarchy from disk
- (C) Delete `~/.claude/CLAUDE.md` so only project-level configuration can load
- (D) Ask Claude in the session to repeat the team's API conventions back

Answer: (A) Run `/memory` in the session to list the loaded memory files

EXPLANATION

`/memory` is the diagnostic command for the configuration hierarchy: it shows which user-level, project-level, and directory-level memory files the current session has loaded, which directly confirms or rules out the suspected loading difference.

Why the other options are wrong

- **B.** `/compact` summarises the conversation to free context; it does not reload or report configuration files. CLAUDE.md content survives compaction, so this neither diagnoses nor fixes a loading difference.
- **C.** Deleting user-level configuration is a destructive guess. It might mask the symptom, but it destroys the developer's personal setup without ever confirming what the failing sessions were loading.
- **D.** The model can paraphrase conventions from training or partial context, so a fluent answer does not prove the files loaded. `/memory` reports the loaded files deterministically instead of relying on the model's self-report.

EXAM TIP

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (`/memory` and CLAUDE.md persistence); Claude Code: Memory and CLAUDE.md

22. In a multi-agent research system the coordinator delegates to search and analysis subagents, then invokes a synthesis subagent. Reviewing the synthesis, the coordinator finds two claims with no supporting evidence and one sub-question left unanswered. What is the coordinator's correct next step?

- (A) Assess the synthesis for gaps, re-delegate targeted follow-up queries to fill them, then re-invoke synthesis with the new findings.
- (B) Accept the synthesis as final, since re-running the search and analysis subagents would push the coordinator past its configured iteration budget.
- (C) Tell the synthesis subagent to fill the gaps from its own knowledge, without any new delegation
- (D) Restart the whole pipeline from scratch with a fresh coordinator to avoid contaminated context

Answer: (A) Assess the synthesis for gaps, re-delegate targeted follow-up queries to fill them, then re-invoke synthesis with the new findings.

EXPLANATION

The coordinator owns iterative refinement: assess the synthesis for gaps, re-delegate focused follow-up queries, then re-synthesise. This closes the evidence gaps without discarding the work already done.

Why the other options are wrong

- **B.** Shipping unsupported claims to save iterations defeats the purpose of the review. The iteration cap is a runaway safety net, not a reason to leave known gaps unfilled.
- **C.** The synthesis subagent has no tools or sources to gather new evidence. Asking it to invent the missing support produces exactly the unsourced claims the review is meant to catch; new evidence requires re-delegation.
- **D.** A full restart throws away correct findings and repeats work. Targeted re-delegation of only the missing pieces is the efficient, guide-recommended fix.

EXAM TIP

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

23. Your legal document extraction pipeline uses `tool_use` with `tool_choice` set to 'auto'. The pipeline processes contracts and sometimes receives plain text analysis instead of the structured JSON extraction you need. The team suggests switching to prompt-based JSON with explicit formatting instructions. What is the correct approach?

(A) Switch to prompt-based JSON as suggested, since the model clearly prefers text responses for these documents

(B) Keep `tool_use` but switch `tool_choice` from 'auto' to 'any' to guarantee the model always returns a structured tool call

(C) Keep `tool_use` and set `tool_choice` to force the specific extraction tool by name, guaranteeing structured output on every request.

(D) Add stronger instructions in the system prompt telling the model to always use the extraction tool and never respond with plain text

Answer: (C) Keep `tool_use` and set `tool_choice` to force the specific extraction tool by name, guaranteeing structured output on every request.

EXPLANATION

Forcing a specific tool with `tool_choice {type: 'tool', name: 'extract_contract'}` guarantees the model calls that exact tool every time, eliminating both text-only responses and wrong-tool selection. This is the most reliable approach for guaranteed structured output.

Why the other options are wrong

- **A.** Prompt-based JSON is less reliable than `tool_use` and can produce malformed output. The issue is `tool_choice` configuration, not the `tool_use` mechanism itself.

- **B.** While 'any' guarantees a tool call, it lets the model choose which tool. If you have multiple tools defined, the model might call the wrong one. For a single extraction tool, this works, but the most precise solution is to force the specific extraction tool.

- **D.** Instructions alone cannot guarantee tool use when `tool_choice` is 'auto'. The API-level `tool_choice` parameter is the correct mechanism for enforcing structured output, not prompt instructions.

EXAM TIP

Lesson 4.2: Structured Output with Tool Use (`tool_choice` forcing); Anthropic: Tool Use Documentation

24. A developer extracting the notification subsystem from a monolith faces 12 cross-module dependencies, 3 messaging patterns, and several valid extraction strategies. They start in direct execution mode. After moving 8 files, the chosen approach breaks circular dependencies with the user-profile module. What should they have done differently?

(A) Used direct execution but with more detailed upfront instructions specifying how to handle each dependency

(B) Used plan mode to map the 12 dependencies and evaluate extraction strategies before committing

(C) Used direct execution but processed only 2 files at a time to catch problems earlier

(D) Delegated the entire extraction to a subagent to isolate the risk

Answer: (B) Used plan mode to map the 12 dependencies and evaluate extraction strategies before committing

EXPLANATION

A subsystem with 12 dependencies, multiple messaging patterns, and several valid strategies is a textbook plan mode scenario. Plan mode would have mapped the dependency graph and revealed the circular dependency with user-profile before any files were moved, avoiding costly rework.

Why the other options are wrong

- **A.** The developer did not know about the circular dependency with the user-profile module upfront. More detailed instructions cannot account for undiscovered dependencies. The codebase needed exploration first.

- **C.** Smaller batches in direct execution do not address the fundamental issue: the developer chose a strategy without understanding the full dependency landscape. The circular dependency would still be discovered mid-migration, just slightly sooner.

- **D.** Delegating to a subagent does not change the outcome if the subagent also uses direct execution without exploring dependencies. The mode of execution (plan vs direct) matters more than who executes it.

EXAM TIP

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode for migrations, Recognising complexity)

25. An analytics agent queries Snowflake and receives 40+ columns per row, only 5 of which are relevant to the user's question about quarterly revenue. The agent appends the full result set to context. After three such queries the window is nearly full and follow-up questions fail. What is the most effective fix?

(A) Upgrade to a model with a larger context window so that full result sets can be accommodated across more queries.

(B) Trim tool results to only the relevant columns before appending them to the conversation context.

(C) Store all query results in an external database and have the agent retrieve specific values on demand instead of keeping results in context.

(D) Limit the number of rows returned by each query to reduce the total data volume in context.

Answer: (B) Trim tool results to only the relevant columns before appending them to the conversation context.

EXPLANATION

Tool result trimming removes irrelevant fields before they enter the context window. Keeping only the 5 relevant columns from each 40+ column result set dramatically reduces token consumption, allowing the agent to handle many more queries within the same context budget.

Why the other options are wrong

- **A.** A larger context window postpones the problem but does not solve it. Each full result set still wastes tokens on 35+ irrelevant columns, and the context will eventually fill regardless of size.
- **C.** External storage adds infrastructure complexity without addressing the core issue. The agent still needs some result data in context to reason about it. Trimming irrelevant fields is simpler and directly reduces context consumption.
- **D.** Row limits reduce data volume but may exclude relevant rows. The problem is column-level verbosity (35+ irrelevant columns per row), not row count. Trimming columns preserves all relevant rows whilst eliminating irrelevant fields.

EXAM TIP

Lesson 5.1: Context Window Management (Tool result trimming, Upstream optimisation)

26. An enterprise data platform team is evaluating how to integrate with their existing PostgreSQL database, Slack workspace, and a custom internal approval workflow. A developer proposes building three custom MCP servers. What is the correct approach?

- (A) Build all three custom MCP servers to ensure tight integration with the team's specific requirements and coding standards.
- (B) Use community MCP servers for PostgreSQL and Slack, and build a custom server only for the internal approval workflow that has no community equivalent.
- (C) Skip MCP entirely and use direct API calls from Bash for all three integrations to avoid the overhead of running MCP servers.
- (D) Use community MCP servers for all three integrations, adapting the internal approval workflow to fit an existing community server's interface.

Answer: (B) Use community MCP servers for PostgreSQL and Slack, and build a custom server only for the internal approval workflow that has no community equivalent.

EXPLANATION

Community servers should be evaluated first for standard integrations. PostgreSQL and Slack have well-maintained community MCP servers. The internal approval workflow is team-specific with no community equivalent, making it the only justified custom build.

Why the other options are wrong

- **A.** Building custom servers for PostgreSQL and Slack is unnecessary when well-maintained community servers already exist for these standard integrations. Custom builds should be reserved for genuinely unique workflows.
- **C.** Direct API calls bypass the MCP tool interface, losing the benefits of structured tool descriptions, standardised error handling, and agent-native integration. MCP servers provide a consistent interface that agents understand natively.
- **D.** Forcing a custom internal workflow into a community server's interface leads to awkward abstractions and missing functionality. The community-first principle applies to standard integrations, not to genuinely unique team-specific workflows.

EXAM TIP

Lesson 2.4: MCP Server Integration (Build-vs-use); Claude Code: MCP Server Configuration

27. Your CI/CD code review system analyses pull requests averaging 8-12 files. Reviews are thorough on the first few files but become increasingly superficial on later files, sometimes missing obvious issues. What architectural change best addresses this pattern?

- (A) Randomise the file order before each review run so a different subset of files receives the thorough early-pass attention each time.
- (B) Increase the model's context window to give it more space to analyse all files
- (C) Split the review into per-file passes so each file gets dedicated attention, then a cross-file integration pass.
- (D) Add a second full-PR review pass and merge findings from both passes

Answer: (C) Split the review into per-file passes so each file gets dedicated attention, then a cross-file integration pass.

EXPLANATION

Per-file analysis ensures each file receives consistent, thorough attention. The cross-file integration pass then catches issues that span multiple files, such as inconsistent interfaces or data flow problems. This directly solves attention dilution.

Why the other options are wrong

- **A.** Randomising order means different files get superficial treatment each time, but does not ensure all files receive thorough review. The root cause - attention dilution - remains.
- **B.** Attention dilution is not a context window size problem. The model has enough space but distributes attention unevenly across many files.
- **D.** A second pass on the full PR suffers from the same attention dilution. Both passes will likely be thorough on early files and superficial on later ones.

EXAM TIP

Lesson 4.6: Multi-Instance Review and Output Validation (Multi-pass review, Why context windows do not fix it)

28. A web search agent has 9 tools: `web_search`, `url_fetch`, `html_parse`, `pdf_extract`, `image_ocr`, `translate`, `summarise`, `keyword_extract`, `sentiment_analysis`. In testing it frequently calls `summarise` and `sentiment_analysis` when it should only fetch raw data. How should the architect fix this?

- (A) Add system prompt instructions telling the web search agent to only use data fetching tools and ignore analysis tools
- (B) Reduce the web search agent to the 4-5 data-fetching tools and move the analysis tools to the specialist agents.
- (C) Keep all 9 tools but implement `PreToolUse` hooks that block the web search agent from calling `summarise` and `sentiment_analysis`
- (D) Merge the web search and synthesis agents into a single agent since they share some tools

Answer: (B) Reduce the web search agent to the 4-5 data-fetching tools and move the analysis tools to the specialist agents.

EXPLANATION

The recommended maximum is 4-5 tools per agent. The web search agent has 9 tools, causing it to use tools outside its intended role. Reducing to data-fetching tools enforces scoped access. The analysis tools (`summarise`, `sentiment_analysis`, `keyword_extract`) belong with the synthesis or analysis agents.

Why the other options are wrong

- A. Prompt instructions are probabilistic and the agent is already misusing tools. The architectural fix is to remove the tools it should not have, providing deterministic enforcement through scoped tool access.
- C. Using hooks to block tool access is an over-engineered workaround for what should be a configuration fix. The proper solution is to remove the tools from `allowedTools`, not to provide them and then block their use.
- D. Merging agents violates the principle of specialised subagents. The solution is to separate tools by role, not to combine agents. Merging would create an even larger tool set on a single agent.

EXAM TIP

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access (4-5 tools)); Anthropic: Claude Agent SDK Overview

29. A CI/CD pipeline runs three Claude Code steps in sequence: (1) generate a changelog from git commits, (2) review the changelog for accuracy, and (3) check for breaking changes. The team notices that step 2 never flags inaccuracies and step 3 misses obvious breaking changes that the changelog omits. What is the root cause?

- (A) The three steps share session context, so steps 2 and 3 inherit step 1's reasoning instead of judging the changelog
- (B) The `CLAUDE.md` file does not contain changelog formatting standards, so the review step has no criteria to evaluate against
- (C) The `-p` flag is not being used, causing each step to wait for interactive input
- (D) The steps need to use `--output-format json` so that each step can parse the previous step's structured output

Answer: (A) The three steps share session context, so steps 2 and 3 inherit step 1's reasoning instead of judging the changelog

EXPLANATION

When review and analysis steps share context with the generation step, they inherit the original reasoning and are less likely to identify gaps or errors. Independent sessions force each step to evaluate the changelog from scratch without bias from the generation reasoning. Session context isolation is critical for CI/CD review pipelines.

Why the other options are wrong

- B. Missing formatting standards would affect the quality of the generated changelog, not the review's ability to catch inaccuracies. The pattern of reviews consistently missing issues points to shared-context bias, not missing criteria.
- C. If `-p` were missing, the pipeline would hang rather than produce output that misses issues. The steps are producing output (the changelog is generated, reviews complete), but the reviews are ineffective.
- D. Output format affects how results are structured, not whether reviews are thorough. JSON output would help with parsing but would not address the fundamental issue of review steps retaining generation context bias.

EXAM TIP

Lesson 3.6: CI/CD Integration (Session context isolation)

- 30.** A moderation classification tool includes a `detected_patterns` array where the model lists specific patterns it identified (e.g. 'repeated slur targeting ethnicity'). Validation checks whether the detected patterns match the assigned category, and on mismatch retries with feedback like 'You classified this as spam but detected patterns of hate speech targeting ethnicity, please re-evaluate.' What is the primary benefit of the `detected_patterns` field in this workflow?
- (A) It lets the system auto-correct the category by overriding the model's classification with a rule-based pattern match
 - (B) It provides an auditable evidence trail showing which specific content features drove each moderation decision
 - (C) It lets validation detect reasoning inconsistencies and feed back targeted errors the model uses to self-correct.
 - (D) It increases classification accuracy by forcing the model to identify specific patterns before assigning a category

Answer: (C) It lets validation detect reasoning inconsistencies and feed back targeted errors the model uses to self-correct.

EXPLANATION

The `detected_patterns` field externalises the model's reasoning into structured data that can be programmatically validated against the conclusion. When patterns and category are inconsistent, the specific error feedback ('you detected hate speech patterns but classified as spam') gives the model actionable information to self-correct. This is the core benefit in a retry-with-error-feedback workflow.

Why the other options are wrong

- **A.** The system does not override the model's classification. It feeds back the inconsistency and asks the model to re-evaluate. The model may correct its category or its patterns - the point is self-correction, not rule-based override.
- **B.** Auditability is a secondary benefit. The primary benefit in this validation/retry workflow is enabling the system to detect and feed back reasoning inconsistencies for self-correction.
- **D.** While structured reasoning fields can improve accuracy, this framing misses the retry workflow context. The primary benefit is enabling programmatic validation of reasoning consistency and targeted feedback for self-correction.

EXAM TIP

Lesson 4.4: Validation, Retry, and Feedback Loops (`detected_patterns` fields, Self-correction flow)

- 31.** A coordinator spawns a research subagent to investigate market trends. The subagent returns detailed findings. The coordinator then needs to pass these findings to a synthesis subagent for report writing. However, the synthesis subagent produces a report that contradicts the research findings. What is the most likely cause?
- (A) The synthesis subagent has a conflicting system prompt that overrides the research data
 - (B) The coordinator passes a summary instead of the full structured research output, so the synthesis agent fills gaps from its training data.
 - (C) The synthesis subagent has its own web search tool and keeps pulling contradictory information from the open web instead of the research it was handed.
 - (D) The two subagents are using different model versions with different training data

Answer: (B) The coordinator passes a summary instead of the full structured research output, so the synthesis agent fills gaps from its training data.

EXPLANATION

Subagents do not share memory. The coordinator must explicitly pass the complete research output to the synthesis agent. If the coordinator only passes a summary, the synthesis agent has no access to the detailed findings and will rely on its training data to fill gaps, potentially contradicting the actual research.

Why the other options are wrong

- **A.** While possible, conflicting system prompts would produce consistently wrong outputs across all requests. The more common cause is that the coordinator is not passing the full research output, so the synthesis agent fills gaps with its own training data.
- **C.** A synthesis subagent should not have web search tools; that violates scoped tool access. The real issue is that the full research output is not being passed to it, not that it is searching the web.
- **D.** Model version differences would not cause contradictions with explicitly provided research data. The issue is that the research data is not being fully passed to the synthesis agent, not that the agents have different base knowledge.

EXAM TIP

Lesson 1.3: Subagent Invocation and Context Passing (Context passing, Structured metadata format)

32. A multi-agent system runs a 45-minute deep research workflow. Around the 30-minute mark, the coordinator's synthesis quality drops: it refers to 'the key findings' instead of specific statistics it cited earlier, and misattributes a claim from the financial subagent to the news subagent. No token limits or errors have been hit. What is the most likely diagnosis and correct mitigation?

- (A) The model is experiencing 'fatigue' from a long session and needs a cooldown period before continuing
- (B) Context degradation: early results are buried deep in a long context. Consolidate key findings into a structured block near the end.
- (C) The token limit has been silently exceeded and the API is truncating early messages. Upgrade to a larger context window model
- (D) The subagents are returning inconsistent data, confusing the coordinator. Add data validation to each subagent's output before it reaches the coordinator

Answer: (B) Context degradation: early results are buried deep in a long context. Consolidate key findings into a structured block near the end.

EXPLANATION

Context degradation occurs when important information is buried deep in a long conversation. The model's attention to early content diminishes as new content accumulates. Consolidating key findings into a recent, structured block keeps them in the model's effective attention window without losing the specifics.

Why the other options are wrong

- **A.** LLMs do not experience fatigue. Each inference is independent. The degradation is caused by context window dynamics, not model tiredness.
- **C.** The question states no token limits have been hit. Context degradation occurs well before token limits are reached - it is an attention quality issue, not a capacity issue.
- **D.** The coordinator previously cited these same findings correctly earlier in the session. The issue is not data quality from subagents but the coordinator's degrading attention to that data over time.

EXAM TIP

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation, Summary injection)

33. A documentation team must update architecture guides across 25 microservice directories after a platform migration. Each guide requires reading source, checking dependencies, and rewriting affected sections. Working in direct execution mode, a team member updates three services before noticing the results are inconsistent: some guides still mention the old platform, others use inconsistent terminology for the new one. What went wrong and what is the correct approach?

- (A) The team member should have used plan mode to set a consistent update strategy and terminology before executing across all 25 services
- (B) Direct execution was correct but should have been done in a single continuous session to maintain consistency through conversation context
- (C) The team member should have created a custom slash command that hardcodes the exact text replacements for each service
- (D) The team member should have used the Explore subagent to discover all affected files first, then made all changes in a single batch

Answer: (A) The team member should have used plan mode to set a consistent update strategy and terminology before executing across all 25 services

EXPLANATION

A 25-service migration with consistency requirements is a multi-step, cross-file operation where planning is essential. Plan mode would produce an explicit strategy covering terminology, update order, and validation criteria before any changes are made, preventing the inconsistency that emerged from ad hoc execution.

Why the other options are wrong

- **B.** A single session updating 25 services would exhaust the context window, causing later updates to lose awareness of earlier decisions. Context degradation would worsen the inconsistency problem, not solve it.
- **C.** Architecture guides require contextual understanding of each service's code and dependencies - not mechanical text replacement. A slash command cannot adapt to the unique structure of each guide.
- **D.** Discovery alone does not solve the consistency problem. Without an explicit plan defining terminology and update patterns, batch execution would produce the same inconsistencies faster.

EXAM TIP

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode for consistency, Decision framework)

34. A coordinator delegates to a data collection agent (`web_search`, `database_query`) and an analysis agent (`calculate`, `chart_generate`). In testing, the analysis agent calls `web_search` directly to fetch extra context, bypassing the data collection agent. What is the architectural violation and how should it be fixed?

- (A) The analysis agent's `allowedTools` includes `web_search`; remove it so only the data collection agent has it.
- (B) Add a system prompt instruction telling the analysis agent not to use `web_search` even though it has access
- (C) Allow the analysis agent to keep `web_search` access since it sometimes needs additional context for better analysis
- (D) Implement a `PostToolUse` hook that logs when the analysis agent uses `web_search` for monitoring purposes

Answer: (A) The analysis agent's `allowedTools` includes `web_search`; remove it so only the data collection agent has it.

EXPLANATION

Each agent should only have tools relevant to its specific role. The analysis agent should have `calculate` and `chart_generate`, not `web_search`. If the analysis agent needs additional data, it should request it through the coordinator, which then delegates to the data collection agent. This enforces the scoped tool access principle.

Why the other options are wrong

- **B.** Prompt instructions are probabilistic. The agent is already occasionally using `web_search` despite the architectural intent. The fix is to remove the tool from `allowedTools`, providing deterministic enforcement rather than hoping the prompt works.
- **C.** This violates scoped tool access and bypasses the data collection agent's role. If the analysis agent needs more data, the correct flow is: analysis agent signals the coordinator, coordinator delegates to the data collection agent, results flow back through the coordinator.
- **D.** Logging the violation does not prevent it. The architectural fix is to remove `web_search` from the analysis agent's `allowedTools`, not to monitor its misuse.

EXAM TIP

Lesson 1.4: Workflow Enforcement and Handoff (Scoped tool access); Anthropic: Claude Agent SDK Overview

35. A documentation team wants to archive the full conversation transcript to a log file every time Claude Code runs `/compact`, so that context lost during compaction can be reviewed later. Which hook configuration achieves this?

- (A) A `PostToolUse` hook on `Write` that snapshots the transcript whenever a file is written, on the assumption `/compact` will eventually run
- (B) A `PreCompact` hook that writes the current transcript to a timestamped log file before `/compact` summarises the conversation
- (C) A `PreToolUse` hook configured on a built-in 'Compact' tool, matching tool name 'Compact'
- (D) A `PostToolUse` hook on all tools that appends each tool result to a running log file, creating a continuous archive

Answer: (B) A `PreCompact` hook that writes the current transcript to a timestamped log file before `/compact` summarises the conversation

EXPLANATION

`PreCompact` is a first-class hook event in Claude Code that fires immediately before `/compact` (or auto-compaction) runs. The hook receives the full pre-compaction transcript path and can copy or process it before summarisation discards detail. This is the documented mechanism for exactly this use case.

Why the other options are wrong

- **A.** `Write` and compaction are unrelated. `PostToolUse` on `Write` would fire on every file write whether or not `/compact` is about to run, producing noise rather than a transcript at the right moment. The dedicated `PreCompact` event is the correct hook.
- **C.** There is no 'Compact' tool exposed to `PreToolUse`. Compaction is a Claude Code lifecycle operation, not a model-invoked tool, so it has its own dedicated `PreCompact` event rather than being hooked via `PreToolUse`.
- **D.** Logging every tool result creates enormous log files and does not capture conversation context (reasoning, planning, user messages). The targeted `PreCompact` event captures the full transcript at the critical moment without per-tool noise.

EXAM TIP

Claude Code: Hooks

36. A user reports a post that contains both potentially defamatory text and an embedded image that may violate graphic content policies. The coordinator receives this report as a single moderation request. What is the correct delegation strategy?

- (A) Send the entire report to the text classifier first, then forward its output to the image analyser so the image step runs only after the text step has finished.
- (B) Send the entire report to whichever subagent handles the more severe category
- (C) Route the text to the text classifier and the image to the image analyser in parallel, then aggregate both results for the final decision.
- (D) Create a new combined text-and-image subagent specifically for multi-modal reports

Answer: (C) Route the text to the text classifier and the image to the image analyser in parallel, then aggregate both results for the final decision.

EXPLANATION

The coordinator should decompose multi-concern requests into independent subtasks. Text defamation and image policy violations are separate analysis dimensions that can be evaluated in parallel. The coordinator aggregates the results to make a unified moderation decision.

Why the other options are wrong

- **A.** Sequential processing creates unnecessary latency when the two concerns are independent. The text and image analyses do not depend on each other and can run in parallel.
- **B.** Severity cannot be determined until both analyses are complete. Routing to a single subagent also means one concern goes unanalysed.
- **D.** Creating a new combined subagent duplicates capability that already exists in the specialist subagents. The coordinator's role is to decompose and delegate, not to spawn new agents for every combination of concerns.

EXAM TIP

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning); Lesson 1.4: Workflow Enforcement and Handoff (Multi-concern request handling)

37. A data-analysis agent connects to an MCP server that exposes 40 database tables. Before almost every query, the agent makes several exploratory tool calls to discover which tables exist and what columns they hold, adding latency and token cost to each task. The server author wants to remove this discovery overhead. What is the most effective change?

- (A) Expose the table catalogue and column schemas as MCP resources for upfront visibility.
- (B) Add a `describe_schema` tool and instruct the agent in its system prompt to call it before every query.
- (C) Enlarge the agent's context window so it can retain the results of the discovery calls across turns.
- (D) Reduce the server to the five most frequently queried tables so there is less to discover.

Answer: (A) Expose the table catalogue and column schemas as MCP resources for upfront visibility.

EXPLANATION

MCP resources expose content catalogues such as database schemas, documentation hierarchies, and issue summaries, giving the agent visibility into available data upfront. The agent no longer needs to call `list_tables` and then `describe_table` for each table, which removes the discovery overhead entirely.

Why the other options are wrong

- **B.** This keeps an exploratory tool call on every task, which is the exact overhead the author wants to remove. Resources present the catalogue without a per-task call.
- **C.** A larger context window does not stop the agent making the discovery calls; it still pays the latency and token cost on each task and merely stores the results.
- **D.** This discards capability the agent legitimately needs and still leaves discovery calls for the remaining tables. It treats the symptom, not the discovery mechanism.

EXAM TIP

Lesson 2.4: MCP Server Integration (MCP resources); Model Context Protocol: Resources

38. Claude Code generates API reference pages for 150 endpoints across three categories: payment processing (30 endpoints, high regulatory risk), internal tooling (80 endpoints, low risk), and public data queries (40 endpoints, moderate risk). The team cannot manually review all 150. How should they structure human review?

- (A) Randomly sample 15% of all endpoints (approximately 23 pages) for review, ensuring a representative cross-section
- (B) Use stratified sampling: review 100% of payment docs, 10% of public data query docs, and 5% of internal tooling docs.
- (C) Review only the payment processing endpoints since they are the highest risk, and trust Claude Code's output for the remaining 120 endpoints
- (D) Have Claude Code self-assess each generated page with a confidence score and only review pages where confidence falls below 80%

Answer: (B) Use stratified sampling: review 100% of payment docs, 10% of public data query docs, and 5% of internal tooling docs.

EXPLANATION

Stratified sampling allocates review effort proportional to risk. Payment processing documentation has regulatory implications warranting full review. Public data queries affect external users and deserve moderate scrutiny. Internal tooling has the lowest blast radius and can tolerate the lightest review. This achieves thorough coverage of critical content within a manageable review budget.

Why the other options are wrong

- **A.** Uniform random sampling treats all endpoints equally, but payment processing endpoints carry much higher risk. A 15% sample might only include 4-5 payment endpoints, inadequately covering the highest-risk category.
- **C.** Completely skipping review for 120 endpoints risks undetected errors in public-facing documentation. Even low-risk internal tooling docs should have some sample review to catch systematic generation issues that might affect all categories.
- **D.** LLM self-reported confidence is poorly calibrated. Claude Code may report high confidence on pages with subtle factual errors (e.g., incorrect parameter types, wrong default values) because the generated text is fluent and internally consistent.

EXAM TIP

Lesson 5.5: Human Review and Confidence Calibration (Reviewer capacity, Stratified sampling)

39. Claude Code is auditing API documentation coverage across a large codebase. After exploring 15 modules, the agent produces vague references like 'the authentication module follows standard patterns' instead of citing specific class names and method signatures it discovered earlier. `/compact` has already been used once. What is the most effective next step?

- (A) Run `/compact` again to further reduce context usage and continue the exploration in the same session.
- (B) Delegate remaining module explorations to subagents, giving each a scratchpad summary of findings so far.
- (C) Start a fresh session and re-explore all 15 modules more efficiently to rebuild the context with only essential findings.
- (D) Increase the model's temperature to encourage more detailed and specific outputs instead of vague pattern references.

Answer: (B) Delegate remaining module explorations to subagents, giving each a scratchpad summary of findings so far.

EXPLANATION

Subagent delegation isolates verbose exploration output from the main agent's context. The scratchpad file persists specific findings (class names, method signatures) across context boundaries. The main agent retains enough context for coordination without being overwhelmed by discovery output.

Why the other options are wrong

- **A.** Running `/compact` again may help marginally, but the fundamental problem is that the agent's context has filled with verbose discovery output from 15 modules. Compacting alone does not restore the specific findings that have been lost to context degradation.
- **C.** Re-exploring 15 modules wastes significant time and effort. The specific findings from earlier exploration are lost. A scratchpad file would have preserved them, and subagent delegation prevents the problem going forward.
- **D.** Temperature affects output randomness, not the model's ability to recall specific findings from earlier in a long context. The problem is context degradation from verbose output accumulation, not generation settings.

EXAM TIP

Lesson 5.4: Codebase Exploration and Context Degradation (Subagent delegation, Scratchpad files)

40. A docs team maintains three content types in `docs/api/`, `docs/architecture/`, and `docs/runbooks/`, each with distinct formatting, terminology, and required sections. They want Claude Code to apply the correct standards automatically when editing any docs file, without loading all three rule sets in every session. What is the correct configuration architecture?

- (A) Place all three sets of rules in the root `CLAUDE.md` with clear section headings so Claude Code can identify which rules apply
- (B) Create three rule files in `.claude/rules/` with YAML frontmatter paths targeting each directory: paths: `['docs/api/**']` for API rules, paths: `['docs/architecture/**']` for architecture rules, and paths: `['docs/runbooks/**']` for runbook rules
- (C) Place a separate `CLAUDE.md` file in each of `docs/api/`, `docs/architecture/`, and `docs/runbooks/` with the type-specific rules
- (D) Create three custom skills (`/api-docs`, `/arch-docs`, `/runbook-docs`) and require writers to invoke the correct one before editing

Answer: (B) Create three rule files in `.claude/rules/` with YAML frontmatter paths targeting each directory: paths: `['docs/api/**']` for API rules, paths: `['docs/architecture/**']` for architecture rules, and paths: `['docs/runbooks/**']` for runbook rules

EXPLANATION

Path-scoped rules in `.claude/rules/` with glob patterns load only when editing files in the matching directory. Each documentation type gets its own rules loaded automatically and exclusively, conserving context tokens and eliminating the risk of applying incorrect standards.

Why the other options are wrong

- **A.** Root CLAUDE.md loads for every session. Including all three rule sets wastes context tokens when editing only one documentation type. The model must also correctly identify which section applies, adding an unnecessary point of failure.
- **C.** Directory-level CLAUDE.md files load based on the working directory hierarchy, but Claude Code sessions typically operate from the repository root. These directory-level files would only load when the agent is directly working within that specific directory, which is less reliable than path-scoped rules that match by file pattern regardless of working directory.
- **D.** This scenario explicitly requires writers to invoke the right skill manually, which depends on human memory and discipline - the wrong pick silently applies the wrong standards. Even if the skills added a paths frontmatter to auto-activate, they'd still load as a task-style workflow rather than as always-in-context guidance. Path-scoped rules in .claude/rules/ are the correct choice: they load into context automatically when Claude reads a matching file, with no human step in the loop.

EXAM TIP

Lesson 3.3: Path-Specific Rules (Practical rule examples, Path-specific rules)

- 41.** A team wants their CI pipeline to run Claude Code for both PR code review and automated test generation. The PR review should output structured JSON for integration with their review dashboard, while the test generation should produce standard text output. How should the two pipeline steps be configured?

- (A) Run both steps with -p and configure the review dashboard to parse plain text output from both steps
- (B) Run the review step with -p --output-format json and the test generation step with -p only, as separate non-interactive invocations
- (C) Run both steps in a single Claude Code session with -p, using different prompts to request different output formats
- (D) Run both steps with --output-format json and have the test generation step extract code from the JSON response

Answer: (B) Run the review step with -p --output-format json and the test generation step with -p only, as separate non-interactive invocations

EXPLANATION

Each step uses -p for non-interactive mode. The review step adds --output-format json for structured output that the dashboard can parse reliably. The test generation step uses default text output since it produces code files, not structured data. Running them as separate invocations ensures session context isolation.

Why the other options are wrong

- **A.** Plain text output requires fragile parsing logic in the review dashboard. The --output-format json flag provides structured, machine-readable output that is far more reliable for programmatic integration.
- **C.** Running both steps in a single session violates session context isolation. The review step would retain reasoning context from test generation, reducing review effectiveness. Each step should be an independent invocation.
- **D.** While this would work technically, it adds unnecessary complexity to the test generation step. Using JSON output format only for the step that needs structured parsing (review) is the cleaner approach.

EXAM TIP

Lesson 3.6: CI/CD Integration (Structured output, -p flag); Claude Code: Headless / CLI Reference

- 42.** A docs team asks Claude Code to audit documentation coverage across a 500,000-line codebase. After reading 30 files via the Read tool, the conversation is approaching the context limit and earlier tool results will be summarised away by auto-compaction. The agent still needs that earlier information to complete the audit. What is the best strategy?

- (A) Increase the context window size so that all 30+ file contents can be held simultaneously without trimming
- (B) Have the agent write structured findings to a scratchpad after each file, then read it for the final audit.
- (C) Process all files in a single Read tool call by passing a glob pattern, so the results arrive in one untrimmed block
- (D) Run /compact after every 10 files to free up context space for the next batch of file reads

Answer: (B) Have the agent write structured findings to a scratchpad after each file, then read it for the final audit.

EXPLANATION

A scratchpad file persists on disk and can be re-read at any time, unlike in-context tool results that are trimmed as new results arrive. By extracting and recording the essential findings (not the full file contents) after each file read, the agent decouples its knowledge from context window limits. The final audit reads the compact scratchpad rather than needing all source files in context.

Why the other options are wrong

- **A.** The context window has a fixed maximum size. A 500,000-line codebase cannot fit entirely in context regardless of configuration. Tool result trimming exists specifically because holding all file contents simultaneously is not feasible.
- **C.** The Read tool reads individual files, not glob patterns. Even if multiple files could be read at once, a single massive result block would itself be trimmed or would consume the entire context budget, preventing the agent from reasoning about the results.
- **D.** /compact summarises the conversation, which may discard the specific findings from earlier file reads. The problem is preserving information, not just freeing space. Compacting without first persisting findings to disk loses the very data the audit needs.

EXAM TIP

Lesson 5.4: Codebase Exploration and Context Degradation (Scratchpad files, /compact behaviour)

43. A CI pipeline validates that generated API docs match the codebase. It runs `claude -p 'Verify that all public API endpoints in src/api/ have corresponding documentation in docs/api/'` with `--output-format json`. The check consistently reports 100% coverage, yet manual audits regularly find undocumented endpoints. What is the most likely cause?
- (A) The `--output-format json` flag corrupts the analysis results, causing false positives
 - (B) Claude Code is doing a shallow pattern match, not a deep semantic comparison, and the prompt lacks validation criteria
 - (C) The `-p` flag prevents Claude Code from reading files, so it generates a plausible-sounding report without actual file access
 - (D) The CI pipeline needs a separate review step where a second Claude Code instance verifies the first instance's findings

Answer: (B) Claude Code is doing a shallow pattern match, not a deep semantic comparison, and the prompt lacks validation criteria

EXPLANATION

Without explicit validation criteria (e.g., 'for each exported function in `src/api/`, check that `docs/api/` contains a section with the function name, parameters, and return type'), the model may perform a shallow check. Adding structured criteria, expected output format, and examples of what constitutes a gap produces reliable verification.

Why the other options are wrong

- **A.** `--output-format json` controls the output structure, not the analysis logic. It formats results as JSON for downstream parsing but does not affect the accuracy of the verification itself.
- **C.** The `-p` flag runs Claude Code in non-interactive (pipe) mode. It does not restrict file access - Claude Code retains full access to its built-in tools (Read, Grep, Glob) for reading files.
- **D.** Adding a second instance to review the first adds latency and cost without addressing the root cause. If the first instance's prompt lacks clear validation criteria, the second instance reviewing the same vague output will draw the same flawed conclusions.

EXAM TIP

Lesson 3.6: CI/CD Integration (Structured output and validation, CLAUDE.md for CI context)

44. An agent receives 200 rows of Snowflake financial data and places it mid-prompt, between system instructions and the user's follow-up question. Asked to identify the row with the highest margin, the agent picks the 45th row (margin 32%) over the 142nd row (margin 47%). What cognitive bias is affecting the agent?
- (A) Recency bias - the agent prioritises data appearing near the end of the context window.
 - (B) The lost-in-the-middle effect: the agent favours the start and end, missing the row buried in the middle.
 - (C) Token limit truncation - the result set exceeds the context window and rows beyond the 100th are silently dropped.
 - (D) Anchoring bias - the agent fixates on the first high-margin row it encounters and stops scanning the rest.

Answer: (B) The lost-in-the-middle effect: the agent favours the start and end, missing the row buried in the middle.

EXPLANATION

The lost-in-the-middle effect causes LLMs to attend disproportionately to content at the start and end of long sequences, with reduced attention to material in the middle. Row 142 (the correct answer) sits deep in the middle of the 200-row result set, making it prone to being overlooked.

Why the other options are wrong

- **A.** Recency bias would favour rows near the end of the data, not the 45th row. The 45th row is in the early portion of the result set, which is consistent with the lost-in-the-middle effect, not recency bias.
- **C.** If rows were truncated, the agent would not have access to row 142 at all and would report the highest margin from the visible rows. The agent identified a specific wrong row (45th), indicating all data is present but attention is uneven.
- **D.** Anchoring bias is a human cognitive bias, not a well-documented LLM failure mode for structured data scanning. The observed behaviour - accurate recall at the start and end, poor recall in the middle - is the hallmark of the lost-in-the-middle effect.

EXAM TIP

Lesson 5.1: Context Window Management (Lost in the middle)

45. A consulting firm's multi-agent research system has a coordinator that always invokes the full pipeline of five subagents (web search, document analysis, data extraction, synthesis, and formatting) for every query, including simple factual lookups that only need web search. This adds unnecessary latency and cost. What is the correct architectural fix?

- (A) Have the coordinator select which subagents to invoke per query, not always the full pipeline.
- (B) Allow subagents to communicate directly with each other so they can skip unnecessary steps in the pipeline
- (C) Add a caching layer so subagents that have no work to do return immediately
- (D) Split into two separate systems: one for simple queries and one for complex queries

Answer: (A) Have the coordinator select which subagents to invoke per query, not always the full pipeline.

EXPLANATION

The coordinator should analyse query requirements and decide which subagents are needed. A simple factual lookup only needs the web search agent, not the full five-agent pipeline. Dynamic selection reduces latency and cost for straightforward requests.

Why the other options are wrong

- **B.** Direct subagent communication violates the hub-and-spoke architecture. All communication must flow through the coordinator. The fix is smarter coordinator routing, not bypassing the coordinator.
- **C.** Caching does not address the architectural issue. Invoking subagents that are not needed adds API call overhead and latency regardless of whether results are cached. The coordinator should not invoke unnecessary subagents in the first place.
- **D.** Maintaining two separate systems creates unnecessary complexity. The coordinator's role is precisely to analyse query requirements and make routing decisions. A single coordinator with dynamic selection handles both cases.

EXAM TIP

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

46. The moderation system handles appeals where users contest a moderation decision. Currently, the same agent that made the original decision re-evaluates the appeal. Appeal overturn rates are suspiciously low. What is the most effective architectural change?

- (A) Add stronger instructions to the appeal handler's prompt requiring it to weigh the user's perspective fairly and set aside its earlier decision.
- (B) Route appeals to a human reviewer who has full authority to overturn automated decisions
- (C) Route appeals to a separate agent instance that cannot see the original reasoning, given only the content and the user's justification.
- (D) Automatically overturn decisions where the user provides any appeal justification to improve user trust

Answer: (C) Route appeals to a separate agent instance that cannot see the original reasoning, given only the content and the user's justification.

EXPLANATION

A fresh agent instance without access to the original reasoning context evaluates the content independently. This avoids confirmation bias from the original decision. The appeal agent sees only the content and the user's argument, enabling a genuinely independent review.

Why the other options are wrong

- **A.** Prompt instructions cannot overcome the bias of an agent reviewing its own decision. The original reasoning context still influences the re-evaluation regardless of instructions.
- **B.** Human review for every appeal does not scale and removes the benefit of automated moderation. Some appeals can be resolved automatically by a separate, unbiased instance.
- **D.** Automatic overturn on any appeal completely undermines moderation. Users who genuinely violated policies would simply appeal every decision.

EXAM TIP

Lesson 1.7: Session State and Resumption (Stale context); Lesson 1.2: Multi-Agent Orchestration (Isolation principle)

47. A moderation team introduces a severity scale ('low', 'medium', 'high', 'critical'). The same post (a user threatening to 'destroy' a competitor's product in a review) is classified as 'high' on some runs and 'low' on others. The prompt defines severities with phrases like 'high means the content is clearly harmful' and 'low means the content is mildly inappropriate'. What should they change?

- (A) Set temperature to 0 to eliminate classification variance across runs
- (B) Replace the prose severity descriptions with a concrete example per level as a calibration anchor.
- (C) Remove the severity scale and use binary classification (violation / not violation) to reduce inconsistency
- (D) Run the classification three times and take the majority vote for severity

Answer: (B) Replace the prose severity descriptions with a concrete example per level as a calibration anchor.

EXPLANATION

Concrete examples anchor each severity level to specific, observable patterns. When the model sees 'destroy a competitor's product' in a review context, it can match against the 'low severity - figurative language' example rather than interpreting 'clearly harmful' differently each run.

Why the other options are wrong

- **A.** Lower temperature reduces randomness but does not fix ambiguous criteria. The model is inconsistent because the severity definitions are vague, not because of sampling variance.
- **C.** Binary classification loses valuable granularity needed for routing decisions (auto-remove vs escalate vs warn). The fix is better severity criteria, not removing severity entirely.
- **D.** Majority voting averages out noise but triples cost and latency without fixing the root cause. With vague criteria, the model may consistently misclassify in the same direction, making voting ineffective.

EXAM TIP

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

48. Project-level `.claude/CLAUDE.md` says 'use 4-space indentation matching the existing codebase.' A senior architect has 'use 2-space indentation' in their user-level `~/ .claude/CLAUDE.md`. In recent sessions the architect's code has come back in 2 spaces and broken the build. The team needs a guarantee that 4-space indentation is applied on every save. What should they do?

- (A) Add a PostToolUse hook that runs the team's formatter after every Write/Edit, so 4-space indentation is enforced regardless of what Claude generates
- (B) Leave the rule in project-level `.claude/CLAUDE.md` - the more specific scope wins on conflicts, so the project rule will override the architect's user-level preference
- (C) Ask the architect to delete their user-level `~/ .claude/CLAUDE.md` so there is no conflict to resolve
- (D) Move the 4-space rule into a `CLAUDE.local.md` at the project root so it is appended last and reads after the user-level file

Answer: (A) Add a PostToolUse hook that runs the team's formatter after every Write/Edit, so 4-space indentation is enforced regardless of what Claude generates

EXPLANATION

Anthropic's memory docs are explicit that `CLAUDE.md` is delivered as a user message with 'no guarantee of strict compliance,' and that 'if two rules contradict each other, Claude may pick one arbitrarily.' The docs themselves point at hooks for this case: hooks 'execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do.' A PostToolUse hook running the formatter is the only option here that gives a hard guarantee.

Why the other options are wrong

- **B.** This is the popular paraphrase, but it's not what Anthropic's docs say. The docs describe a load order (broadest scope to most specific, so project instructions appear in context after user instructions) but explicitly state files are 'concatenated into context rather than overriding each other' and conflicts 'may [be] pick[ed] arbitrarily.' The team has already seen that assumption fail in production.
- **C.** This treats a personal config file as a team problem and doesn't scale - every new teammate would need to police their own home directory, and any future contradiction (from a different file, an `@import`, or a `.claude/rules/` entry) would resurface the same fragility. The fix is to remove reliance on guidance-style config for a rule that must be enforced, not to remove the conflicting file.
- **D.** Load order does append `CLAUDE.local.md` after `CLAUDE.md` within a directory, but the docs are careful to call this 'load order,' not precedence, and warn that conflicts may resolve arbitrarily. `CLAUDE.local.md` is also gitignored, so a team standard cannot live there.

EXAM TIP

Lesson 3.1: `CLAUDE.md` Hierarchy and Scoping (Loading order and conflict handling); Anthropic - How Claude remembers your project (memory docs)

49. The moderation system classifies hate speech accurately for explicit slurs but misses coded language and dog-whistle terms that human moderators easily recognise. The prompt includes detailed written rules about coded language patterns. What intervention would most improve detection of coded hate speech?

- (A) Add a comprehensive dictionary of every known coded term and dog-whistle to the system prompt so the classifier can match posts against the full list.
- (B) Add 2-4 few-shot examples of coded hate speech, with reasoning naming the coded language and the targeted group.
- (C) Add 20+ examples covering every known category of coded hate speech to maximise coverage
- (D) Increase the model's context window so it can consider more of the user's post history for context

Answer: (B) Add 2-4 few-shot examples of coded hate speech, with reasoning naming the coded language and the targeted group.

EXPLANATION

Few-shot examples with reasoning tags teach the model the pattern-recognition process, not just individual terms. Showing the reasoning chain - surface meaning, coded meaning, targeted group, contextual signals - enables the model to generalise to new coded terms it has not seen before.

Why the other options are wrong

- **A.** A static dictionary becomes outdated immediately as coded language constantly evolves. It also bloats the prompt without teaching the model to recognise the underlying patterns of coded speech.
- **C.** Laundry lists of examples dilute attention and teach pattern-matching rather than generalisation. 2-4 targeted examples with reasoning are more effective than 20 examples without reasoning because they teach the analytical

process.

- **D.** Context window size is not the bottleneck. The model fails to recognise coded language in individual posts, which is a prompt engineering issue, not a context limitation.

EXAM TIP

Lesson 4.2: Few-Shot Prompting (Few-shot for nuanced detection); Anthropic: Multishot (Few-Shot) Prompting

- 50.** The moderation team asks a single Claude session to classify a post, then immediately asks the same session to independently review its own classification for quality assurance. The 'review' agrees with the original classification 98% of the time, including cases that human auditors later identify as errors. Why is this self-review ineffective?

(A) The model needs a stronger review prompt with explicit instructions to look for errors in the original classification

(B) The same session retains the model's reasoning, so it stays anchored to its classification; use a fresh independent instance to review.

(C) The model's temperature is too low, producing deterministic agreement - increase temperature for the review pass

(D) Self-review is effective but the 98% agreement rate simply reflects high initial accuracy - the 2% disagreement is the expected error rate

Answer: (B) The same session retains the model's reasoning, so it stays anchored to its classification; use a fresh independent instance to review.

EXPLANATION

Self-review within the same session is fundamentally limited because the model's conversation context includes the original reasoning. It will naturally be anchored to its prior conclusions. An independent instance with no access to the original decision evaluates the content fresh, providing genuine quality assurance.

Why the other options are wrong

- **A.** Stronger instructions do not overcome the fundamental limitation. The same session retains the reasoning context from the original classification, biasing the review toward agreement.

- **C.** Temperature affects sampling randomness, not reasoning independence. Higher temperature may occasionally produce different outputs but does not create genuine independent review. The model is still anchored to its own prior reasoning in the same session.

- **D.** Human auditors found errors in cases where the self-review agreed, proving the 98% agreement does not reflect accuracy. The self-review is confirming errors, not catching them, due to same-session reasoning bias.

EXAM TIP

Lesson 4.6: Multi-Instance Review and Output Validation (Self-review limitation)

- 51.** A consulting firm needs an agent to add a comprehensive test suite to a large legacy codebase. The codebase has no existing tests, unclear dependencies between modules, and the team does not know which areas are most critical. Which task decomposition strategy is most appropriate?

(A) A fixed sequential pipeline that reviews each file in alphabetical order and generates tests for each one

(B) A single-pass analysis that processes the entire codebase at once and generates a complete test plan

(C) Dynamic adaptive decomposition: map the codebase, find the high-impact areas, and adapt the plan as dependencies emerge.

(D) A multi-pass architecture with per-file analysis and a cross-file integration pass, exactly like a standard code review pipeline.

Answer: (C) Dynamic adaptive decomposition: map the codebase, find the high-impact areas, and adapt the plan as dependencies emerge.

EXPLANATION

Adding tests to a legacy codebase with unknown dependencies is an open-ended investigation task. Dynamic adaptive decomposition generates subtasks based on what is discovered at each step - first mapping the structure, then identifying critical areas, then adapting the test plan as dependency relationships emerge.

Why the other options are wrong

- **A.** Fixed sequential pipelines are best for predictable, structured tasks. Adding tests to a legacy codebase with unclear dependencies is an open-ended investigation that requires adaptive exploration to discover which areas are most critical and how dependencies affect test design.

- **B.** A single-pass analysis of a large codebase would suffer from attention dilution, producing inconsistent coverage. Additionally, a single pass cannot adapt to discoveries about dependencies and critical areas that emerge during exploration.

- **D.** Multi-pass architecture solves attention dilution for review tasks, but test generation in a legacy codebase needs adaptive exploration: discovering critical modules and dependencies and prioritising coverage. The decomposition must adapt to findings, not follow a fixed per-file pattern.

EXAM TIP

Lesson 1.6: Task Decomposition Strategies (Dynamic adaptive decomposition, Pattern selection)

52. The data platform team needs to share their Snowflake and PostgreSQL MCP server configurations with all team members whilst allowing individual developers to experiment with a personal API integration server. Where should each configuration be placed?
- (A) Place all three servers in the project-level `.mcp.json` so they are version-controlled and consistent across the team.
 - (B) Place the Snowflake and PostgreSQL servers in the project-level `.mcp.json`, and personal API integration servers in user-level `~/.claude.json`.
 - (C) Place all three servers in `~/.claude.json` and have each developer copy the configuration manually.
 - (D) Place the database servers in environment variables and reference them in both `.mcp.json` and `~/.claude.json`.

Answer: (B) Place the Snowflake and PostgreSQL servers in the project-level `.mcp.json`, and personal API integration servers in user-level `~/.claude.json`.

EXPLANATION

Project-level `.mcp.json` is version-controlled and shared - correct for team-wide database integrations. User-level `~/.claude.json` is machine-specific - correct for personal experimental servers that should not affect other team members.

Why the other options are wrong

- **A.** Personal experimental servers should not be in project-level configuration. Changes to personal servers would create unnecessary merge conflicts and force experimental integrations on the entire team.
- **C.** User-level configuration is not version-controlled and requires manual synchronisation. Team-wide servers belong in project-level `.mcp.json` for consistency and change tracking.
- **D.** Environment variables are used for credentials (connection strings, API keys), not for MCP server configuration. The server definitions themselves belong in `.mcp.json` or `~/.claude.json` depending on scope.

EXAM TIP

Lesson 2.4: MCP Server Integration (Scoping hierarchy); Claude Code: MCP Server Configuration

53. The moderation system's prompt instructs Claude to 'be conservative when moderating and err on the side of caution.' Reviewers find that innocuous posts about cooking with knives, news articles about violence, and fictional war stories are all being flagged as policy violations. What is the root cause and fix?
- (A) The model is too sensitive - lower the temperature to reduce over-flagging
 - (B) Replace 'be conservative' with explicit categorical criteria defining each violation category with concrete examples.
 - (C) Add an allowlist of safe topics (cooking, news, fiction) that should never be flagged
 - (D) Add a second moderation pass that re-reads each flagged post under the same 'be conservative' guidance and strips out the false positives before any action.

Answer: (B) Replace 'be conservative' with explicit categorical criteria defining each violation category with concrete examples.

EXPLANATION

Vague directives like 'be conservative' give the model no basis for distinguishing between genuine violations and legitimate content that mentions sensitive topics. Explicit criteria with examples of both violations and non-violations (news, fiction, educational content) enable consistent, calibrated decisions.

Why the other options are wrong

- **A.** Temperature affects randomness, not the interpretation of moderation criteria. The over-flagging is caused by vague instructions, not sampling behaviour.
- **C.** Allowlists are brittle and impossible to maintain comprehensively. New safe topics will constantly appear. The fix is better criteria that teach the model to distinguish context, not a growing list of exceptions.
- **D.** A second pass using the same vague 'be conservative' criteria will reproduce the same false positives. Fix the criteria first; add verification layers second.

EXAM TIP

Lesson 4.1: System Prompts with Explicit Criteria (Explicit criteria over hedge phrases, Severity calibration)

54. The data platform's `fetch_api` tool calls a third-party pricing API. When the API key has expired, the tool returns `{ "data": [], "status": "ok" }` instead of signalling an authentication failure. The agent tells the user 'No pricing data is available for that product' and moves on. What is the root cause?
- (A) The agent should be instructed via the system prompt to treat empty arrays as errors and retry with exponential backoff.
 - (B) The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data.
 - (C) The third-party API is poorly designed. The platform team should switch to a different pricing provider that returns proper HTTP error codes.
 - (D) The agent's context window is too small to hold the full API response, so it receives a truncated version that appears empty.

Answer: (B) The tool cannot distinguish an access failure (expired key) from a valid empty result, so the agent treats a permission error as no data.

EXPLANATION

The tool returns the same structure for two fundamentally different outcomes: 'I could not access the data' versus 'I accessed the data and found nothing.' The agent cannot distinguish these and accepts the empty result at face value. The tool must return a structured error with the MCP `isError` flag for authentication failures.

Why the other options are wrong

- **A.** Not all empty arrays are errors - a product with no pricing data is a valid empty result. The problem is that the tool masks an authentication failure as a successful empty response. System prompt instructions cannot reliably distinguish the two.
- **C.** Whilst the upstream API behaviour is unhelpful, the platform team controls the MCP tool layer. The tool should detect the authentication failure (even from the upstream response) and translate it into a structured MCP error rather than passing through the misleading response.
- **D.** Context window truncation would not produce a well-formed `{ "data": [], "status": "ok" }` response. The problem is semantic - the tool is returning a misleading success status for an authentication failure.

EXAM TIP

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

- 55.** A multi-agent research system has a coordinator that spawns a web search subagent and a document analysis subagent sequentially across separate API turns. The web search completes in 8 seconds and the document analysis completes in 12 seconds, giving a total latency of 20 seconds. The two subagents are investigating independent topics and do not depend on each other's results. How should the architect reduce this latency?

- (A) Merge the web search and document analysis into a single subagent to reduce coordination overhead
- (B) Have the coordinator emit both Task tool calls in a single response to spawn both subagents in parallel, reducing total latency to roughly 12 seconds
- (C) Allow the web search subagent to directly invoke the document analysis subagent after it finishes, removing the coordinator round-trip
- (D) Use `fork_session` to split the coordinator into two parallel branches, one for each subagent

Answer: (B) Have the coordinator emit both Task tool calls in a single response to spawn both subagents in parallel, reducing total latency to roughly 12 seconds

EXPLANATION

When subagents investigate independent topics, the coordinator can emit multiple Task tool calls in a single response. Both subagents run concurrently, so total latency is determined by the slower one (12 seconds) rather than the sum (20 seconds).

Why the other options are wrong

- **A.** Merging agents violates the principle of specialised subagents with scoped tool access. It also does not reduce total processing time - the merged agent still needs to do both tasks, and loses the benefit of parallel execution.
- **C.** Direct subagent communication violates the hub-and-spoke architecture where all communication flows through the coordinator. This also would not achieve parallel execution - it would still be sequential.
- **D.** `fork_session` is for divergent exploration of the same problem from a shared baseline, not for parallel subagent invocation. Emitting multiple Task tool calls in a single coordinator response is the correct mechanism for spawning concurrent subagents.

EXAM TIP

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning, Task tool)

- 56.** A docs team has Claude Code generate architecture guides by synthesising source code, inline comments, commit messages, and ADRs. After several edits, the guides no longer indicate which statements came from which source. A developer later questions whether a specific architectural constraint in a guide is still valid. What approach preserves source attribution?

- (A) Instruct Claude Code to add footnotes to the generated documentation citing the original source for each statement
- (B) Maintain structured provenance metadata that maps each claim to its source file, line number, and retrieval date.
- (C) Keep all original source files unchanged in a reference directory so developers can manually trace any claim back to its origin
- (D) Version-control the documentation and use `git blame` to trace each line back to the commit that created it

Answer: (B) Maintain structured provenance metadata that maps each claim to its source file, line number, and retrieval date.

EXPLANATION

Structured provenance metadata survives editing because it is stored as data fields (e.g., JSON or YAML frontmatter) separate from the prose. When a developer questions a claim, they can trace it to the exact source file and line number. The retrieval date indicates whether the source was current when the documentation was generated.

Why the other options are wrong

- **A.** Prose footnotes are fragile - they are lost or degraded during subsequent editing passes, summarisation, or reformatting. The team has already experienced this problem through 'several iterations of editing.'

- **C.** Preserving source files without explicit mappings forces developers to manually search through potentially hundreds of files to verify a single claim. This is not scalable and does not indicate which specific source informed which specific documentation statement.
- **D.** git blame traces authorship of documentation lines to commits, not to the source material that informed those lines. A commit message might say 'update architecture guide' without indicating that a specific statement came from ADR-047 or a comment in auth-service/src/middleware.ts.

EXAM TIP

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Attribution preservation, Structured claim-source mappings)

- 57.** A developer has personal code formatting preferences (2-space indentation, trailing commas) that differ from some of the projects they work on. They want Claude Code to apply these preferences across all of their projects by default. Where should these preferences be configured?

- (A) In the project-level `.claude/CLAUDE.md` of each repository they work on
- (B) In `~/.claude/CLAUDE.md`, so they apply across all of the developer's projects
- (C) In a `.claude/rules/` file without YAML frontmatter so it loads for every session
- (D) In a `~/.claude/skills/` file that is invoked before each coding session

Answer: (B) In `~/.claude/CLAUDE.md`, so they apply across all of the developer's projects

EXPLANATION

User-level `CLAUDE.md` (`~/.claude/CLAUDE.md`) applies to all of that developer's projects and is not shared with the team via version control, which is exactly right for personal preferences. Note that `CLAUDE.md` files are concatenated into context rather than strictly overriding one another, so a project's conflicting standard is not guaranteed to win; if a rule must always be enforced, put it in `settings.json` or a hook.

Why the other options are wrong

- **A.** Project-level configuration is shared with the team via version control. Personal formatting preferences should not be imposed on the entire team. Additionally, this would require duplicating the preferences across every project.
- **C.** `.claude/rules/` is project-scoped and version-controlled. Personal preferences should not be committed to project repositories where they would affect all team members.
- **D.** Skills are on-demand workflows invoked for specific tasks, not always-loaded configuration. Formatting preferences need to be applied automatically to every session without manual invocation.

EXAM TIP

Lesson 3.1: `CLAUDE.md` Hierarchy and Scoping (User-level `CLAUDE.md`); Claude Code: Memory and `CLAUDE.md`

- 58.** The coordinator receives a batch of 200 flagged posts to moderate. Currently it processes them sequentially, taking 45 minutes. Each post's moderation is independent - the decision on one post does not affect others. How should the coordinator handle this batch?

- (A) Process all 200 posts in a single API call by concatenating them into one large prompt
- (B) Delegate independent posts to parallel subagent instances, with the coordinator aggregating results as they complete
- (C) Split into fixed batches of 20 posts and process each batch sequentially in a single prompt
- (D) Increase the iteration cap on the agentic loop to allow the agent more time to process all 200 posts

Answer: (B) Delegate independent posts to parallel subagent instances, with the coordinator aggregating results as they complete

EXPLANATION

Since each post's moderation is independent, the coordinator should delegate them to parallel instances. This reduces total processing time from sequential (45 minutes) to roughly the time of the slowest individual moderation, and a failure on one post does not block others.

Why the other options are wrong

- **A.** Concatenating 200 posts into a single prompt causes attention dilution. Quality degrades for posts later in the sequence, and a single failure blocks the entire batch.
- **C.** Batching 20 posts into a single prompt still risks attention dilution across posts within each batch. And sequential batch processing still takes much longer than parallel individual processing.
- **D.** The iteration cap controls how many tool calls the agent can make within a single task, not how many independent tasks it can process. The issue is parallelism, not loop duration.

EXAM TIP

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

Claude CCAF Practice Mock 2 (Official Google Practice Exam - 60Q)

Mock Tests - 60 questions

59. An architect is designing an autonomous agent loop using Claude 3.5 Sonnet to handle customer refunds. The agent must check a database, verify conditions, and execute a payout tool. To prevent the agent from getting stuck in an infinite loop if a tool fails repeatedly, which architectural pattern should be implemented?

- (A) Set the API temperature to 1.0 to introduce randomness in retries.
- (B) Implement an orchestrator-level state tracker that counts sequential identical tool calls and triggers a circuit breaker if a threshold is exceeded.
- (C) Use a smaller model like Claude 3 Haiku for the fallback loop.
- (D) Blindly reduce the `max_tokens` parameter to truncate the execution loop.

Answer: (B) Implement an orchestrator-level state tracker that counts sequential identical tool calls and triggers a circuit breaker if a threshold is exceeded.

EXPLANATION

A stateful tracking layer prevents uncontrolled loops, which are a major stability and cost risk in autonomous agent systems.

60. You are building an application where Claude needs to evaluate its own output against a set of compliance rules before returning it to the user. This is an example of which agentic design pattern?

- (A) Routing/Triage Pattern
- (B) Parallel Tool Execution Pattern
- (C) Critic/Reflexion Pattern
- (D) Single-turn Extraction Pattern

Answer: (C) Critic/Reflexion Pattern

EXPLANATION

Self-correction or validation cycles against rules map directly to the Critic/Reflexion agentic loop design.

61. When building a multi-agent system where a "Supervisor" agent delegates specialized tasks to a "Writer" agent and a "Coder" agent, what is the best practice for managing state between them?

- (A) Pass the full, unedited chat history of all interactions to every agent in every turn.
- (B) Use a centralized orchestrator state machine that extracts relevant outputs from one agent and passes them as cleanly formatted inputs to the next.
- (C) Have the agents communicate directly with each other via raw prompt injections without an intermediate orchestrator.
- (D) Restrict all agents to single-turn completions and disable multi-agent messaging entirely.

Answer: (B) Use a centralized orchestrator state machine that extracts relevant outputs from one agent and passes them as cleanly formatted inputs to the next.

EXPLANATION

State aggregation via an orchestrator controls noise and token spend, preventing cascading context window bloat.

62. An agentic system built on Claude 3.5 Sonnet needs to process an ambiguous user request that might require three separate tool calls. To minimize total turnaround time, the architect wants to use parallel tool calling. What is a critical requirement for the backend application handling this?

- (A) The backend must execute the tools sequentially to avoid confusing the LLM.
- (B) The tool execution layer must support concurrent execution and handle shared state safely without race conditions.
- (C) All tools must accept identical JSON schemas.
- (D) Claude must be forced into a 3-second wait state between tool blocks.

Answer: (B) The tool execution layer must support concurrent execution and handle shared state safely without race conditions.

EXPLANATION

Concurrency management is critical at the application tier when an LLM issues multiple tool targets simultaneously.

- 63.** An engineer notices that an agent using a ReAct (Reasoning + Acting) loop becomes progressively less accurate as the conversation extends to 30+ turns. What is the root cause and the most robust architectural fix?
- (A) Root cause: Model temperature drift. Fix: Dynamically lower temperature to 0 over time.
 - (B) Root cause: Context window dilution and noise accumulation. Fix: Implement a sliding window with a summary layer for older turns.
 - (C) Root cause: Tool definition degradation. Fix: Redefine the tools in every single prompt turn using different names.
 - (D) Root cause: Token limit exhaustion. Fix: Switch the model to an open-source alternative mid-session.
- Answer: (B)** Root cause: Context window dilution and noise accumulation. Fix: Implement a sliding window with a summary layer for older turns.
- EXPLANATION**
- Multi-turn loops accumulate attention noise and redundant tokens. Rolling summaries maintain state cleanly.
- 64.** In an orchestrator-worker pattern, why is it beneficial to use Claude 3 Haiku for the initial classification/routing step rather than Claude 3.5 Sonnet or Opus?
- (A) Haiku has a larger context window than Sonnet or Opus.
 - (B) Haiku provides a significant reduction in Time-to-First-Token (TTFT) and lower token cost for high-volume routing tasks.
 - (C) Haiku natively supports more tool schemas than Sonnet.
 - (D) Haiku eliminates the need for an orchestrator layer entirely.
- Answer: (B)** Haiku provides a significant reduction in Time-to-First-Token (TTFT) and lower token cost for high-volume routing tasks.
- EXPLANATION**
- Triage tasks should minimize TTFT and transaction costs; Haiku is highly optimized for fast categorization.
- 65.** You are designing a routing architecture for an enterprise helpdesk. Simple queries should go to a fast model, while complex reasoning queries go to a premium model. What is the most reliable way to perform this routing?
- (A) Use a rule-based regex parser first; if it fails, use a fast LLM classifier to evaluate intent into distinct categories.
 - (B) Send all queries to both models simultaneously and let the user choose the best response.
 - (C) Use a vector embedding distance match against 1,000,000 documents for every single incoming greeting.
 - (D) Randomly distribute 50% of traffic to each model to balance server load.
- Answer: (A)** Use a rule-based regex parser first; if it fails, use a fast LLM classifier to evaluate intent into distinct categories.
- EXPLANATION**
- Rule-based triage combined with low-cost classification models ensures deterministic routing without unnecessary high-tier model expense.
- 66.** When an autonomous agent using Claude calls a tool, how does the orchestrator signal a failure (e.g., an API timeout) back to Claude so it can self-correct?
- (A) The orchestrator should raise a local code exception and crash the user session.
 - (B) The orchestrator must append a new message block with the role `user` (or a specific tool response block) indicating the failure state or error message.
 - (C) The orchestrator must modify Claude's original system prompt to include the error message.
 - (D) The orchestrator should re-submit the request with a higher temperature without mentioning the error.
- Answer: (B)** The orchestrator must append a new message block with the role `user` (or a specific tool response block) indicating the failure state or error message.
- EXPLANATION**
- Error boundaries are communicated back into the context conversation window via structured `user` or `tool_result` message elements.
- 67.** Which of the following scenarios is uniquely suited for a multi-agent choreography architecture over a single monolithic agent prompt?
- (A) Extracting 5 fields of structured data from a single invoice PDF.
 - (B) A software development workflow requiring separate phases for requirement analysis, code generation, independent code review, and automated unit testing.
 - (C) Translating a short paragraph from English to Spanish.
 - (D) Running a quick mathematical calculation using a calculator tool.
- Answer: (B)** A software development workflow requiring separate phases for requirement analysis, code generation, independent code review, and automated unit testing.
- EXPLANATION**

Sequential phase gates with varying rule sets are the textbook use case for multi-agent isolation architectures.

68. An agent is tasked with writing a complex report. You implement a Plan-and-Solve pattern. How does this differ fundamentally from a standard zero-shot prompt?

- (A) The model is instructed to output an explicit step-by-step execution plan first before executing the sub-tasks.
- (B) The model bypasses the prompt entirely and queries an external planning database.
- (C) The model executes all steps in parallel without considering dependencies.
- (D) It relies exclusively on few-shot examples without structural instructions.

Answer: (A) The model is instructed to output an explicit step-by-step execution plan first before executing the sub-tasks.

EXPLANATION

Plan-and-solve enforces explicit intermediate checkpoint tokens to lay out structural reasoning dependencies before code/text generation.

69. You want an agent to dynamically discover available APIs at runtime rather than hardcoding 50 different tool definitions into the system prompt. How can you architect this cleanly?

- (A) Put all 50 tools into the prompt regardless of token cost, as Claude ignores irrelevant tools.
- (B) Implement a two-step RAG pipeline where a coordinator agent searches a vector index of tool schemas based on the user's query, and only injects relevant tool schemas into the active execution prompt.
- (C) Force Claude to guess the tool names and catch the syntax errors on the client side.
- (D) Use a single generic tool called `call_any_api` that accepts an unvalidated raw URL and payload string.

Answer: (B) Implement a two-step RAG pipeline where a coordinator agent searches a vector index of tool schemas based on the user's query, and only injects relevant tool schemas into the active execution prompt.

EXPLANATION

Dynamic runtime tool-injection via a RAG vector index saves extensive prompt token costs and scales efficiently.

70. What is a major risk of allowing an LLM agent to operate in a completely unconstrained loop (`while True`) without an orchestrator-enforced step limit?

- (A) The model will run out of internal weights and degrade permanently.
- (B) Extreme cost accumulation and infinite loops due to unexpected edge case inputs or unhandled tool errors.
- (C) The API key will automatically rotate and lock the system out.
- (D) The model will start returning responses in the wrong language.

Answer: (B) Extreme cost accumulation and infinite loops due to unexpected edge case inputs or unhandled tool errors.

EXPLANATION

Uncapped automation loops risk massive API billing and unchecked operational errors if edge case inputs loop indefinitely.

71. When designing a human-in-the-loop (HITL) pattern for a high-risk financial trading agent, at what point should the orchestrator pause execution to wait for human authorization?

- (A) Before parsing the user's initial prompt text.
- (B) Immediately after a tool successfully completes an internal read-only database lookup.
- (C) After the model generates a tool call for `execute_wire_transfer` but before the orchestrator runs the underlying execution code.
- (D) Only after the wire transfer has been irreversibly processed by the clearing bank.

Answer: (C) After the model generates a tool call for `execute_wire_transfer` but before the orchestrator runs the underlying execution code.

EXPLANATION

High-risk boundary operations must intercept execution after intent generation but before backend code commitment.

72. An agent needs to synthesize feedback from three different expert agents. Which aggregation technique minimizes conflicting instructions in the final output?

- (A) Appending all three outputs back-to-back without structure.
- (B) Using a final "Synthesizer" or "Editor" agent prompt instructed to reconcile discrepancies and format a unified response.
- (C) Selecting the longest response and discarding the other two.
- (D) Averaging the token lengths of the responses.

Answer: (B) Using a final "Synthesizer" or "Editor" agent prompt instructed to reconcile discrepancies and format a unified response.

EXPLANATION

Aggregating expert outputs requires an explicit processing layer or specialized judge/editor model to resolve instruction conflicts.

73. Why does a "Chain-of-Thought" (CoT) approach improve an agent's success rate when navigating multi-step logic pathways?

- (A) It directly reduces the token cost per generation.
- (B) It allows the model to allocate computational tokens to decompose the problem into manageable steps before generating a conclusion.
- (C) It completely prevents the model from making syntax errors in JSON.
- (D) It increases the throughput speed of the API gateway.

Answer: (B) It allows the model to allocate computational tokens to decompose the problem into manageable steps before generating a conclusion.

EXPLANATION

CoT exposes reasoning trajectories as tokens, allowing self-correcting generation paths before final selections.

74. You are building an agent that must interact with a legacy terminal system. The terminal often returns multi-page unformatted text tables. Which pattern works best?

- (A) Feed the raw unformatted text into the agent and hope it interprets the layout natively.
- (B) Use an intermediate parser tool that converts the unformatted text table into structured JSON or Markdown before passing it to Claude.
- (C) Increase the model's temperature to allow it to "guess" missing columns.
- (D) Convert the entire terminal output into a single giant string and delete all spaces.

Answer: (B) Use an intermediate parser tool that converts the unformatted text table into structured JSON or Markdown before passing it to Claude.

EXPLANATION

Transforming messy infrastructure text structures into structured JSON or Markdown normalizes context tokens for predictable downstream parsing.

75. An enterprise development team is integrating the `claude-code` CLI tool into their localized development workflow. When configuration variables need to be scoped strictly to a specific project repository, where should they be declared?

- (A) In the user's global shell configuration (`~/.bashrc` or `~/.zshrc`).
- (B) Inside a local configuration file located at the root directory of the repository.
- (C) Passed as environment variables via an external cloud vault every time a command runs.
- (D) Hardcoded into the system kernel variables.

Answer: (B) Inside a local configuration file located at the root directory of the repository.

EXPLANATION

Repository-level scoping isolates configuration patterns to prevent accidental environment variables leakage.

76. When executing file system modifications via `claude-code` or an automated codebase assistant, what is the best practice for ensuring safe code merges?

- (A) Allow the agent to write directly to the main or production branch without constraints.
- (B) Have the tool operate on a dedicated feature branch, and enforce a human-reviewed Pull Request (PR) workflow with automated CI pipeline testing.
- (C) Delete all unit tests so the agent's code never triggers a test failure flag.
- (D) Disable git tracking entirely while the agent is writing code.

Answer: (B) Have the tool operate on a dedicated feature branch, and enforce a human-reviewed Pull Request (PR) workflow with automated CI pipeline testing.

EXPLANATION

Secure coding execution dictates isolation to feature branches validated by automated continuous integration loops and human checks.

77. Your automated coding workspace assistant keeps modifying files outside the intended module scope. How can you cleanly restrict its file visibility and writing capabilities?

- (A) Rename all other files to hidden files.
- (B) Configure a targeting array or path ignore list (such as `.gitignore` or a dedicated tool configuration file) to restrict file system search tool paths.
- (C) Enforce a maximum token limit of 50 tokens per file read.
- (D) Encrypt the rest of the codebase with an asymmetric key.

Answer: (B) Configure a targeting array or path ignore list (such as `.gitignore` or a dedicated tool configuration file) to restrict file system search tool paths.

EXPLANATION

Scoping target file system rules within path exclusion definitions matches production tool execution boundaries.

78. A development team wants to configure Claude to write automated unit tests for legacy code. To maximize test quality, what context should be provided alongside the target source file?

- (A) The entire git commit history from the last three years.
- (B) The project's testing framework standards, a mock data utility guide, and a few examples of well-written existing tests.
- (C) The compiler source code of the operating system.
- (D) A list of all developer usernames and email addresses.

Answer: (B) The project's testing framework standards, a mock data utility guide, and a few examples of well-written existing tests.

EXPLANATION

Test quality correlates directly with providing in-context execution rules, architecture frameworks, and high-quality baseline few-shot examples.

79. When using Claude for large-scale code refactoring, what is a primary limitation of processing a codebase purely through a standard text-based chat interface compared to a specialized workspace tool?

- (A) Standard chat interfaces cannot understand programming languages other than Python.
- (B) Standard chat interfaces lack automatic file system synchronization, dependency graph tracking, and terminal execution hooks.
- (C) Standard chat interfaces are entirely deterministic and cannot generate creative code.
- (D) Standard chat interfaces enforce a maximum limit of 10 lines of code per response.

Answer: (B) Standard chat interfaces lack automatic file system synchronization, dependency graph tracking, and terminal execution hooks.

EXPLANATION

Standard web completion lacks file-system manipulation trees, workspace terminal execution loops, and git hooks.

80. An architect needs to configure automated linting checks into an LLM-driven coding agent pipeline. Why should linting occur immediately after code generation?

- (A) It reduces the cost of the initial generation block.
- (B) It allows the system to catch and automatically feed syntax/style errors back to the model for self-correction before any human code review occurs.
- (C) It changes the model's architecture from dense to sparse.
- (D) It speeds up the internet connection to the API endpoint.

Answer: (B) It allows the system to catch and automatically feed syntax/style errors back to the model for self-correction before any human code review occurs.

EXPLANATION

Inline linting checks capture syntax patterns immediately, looping errors into context for low-latency self-healing paths.

81. When providing feedback to Claude Code about a compilation failure, which input format yields the fastest and most accurate resolution?

- (A) "The build failed, please fix it."
- (B) The complete, exact compiler error log along with the names and relevant lines of the files that failed to compile.
- (C) A screenshot of the terminal window converted to a base64 string.
- (D) A link to a public forums page discussing a similar error.

Answer: (B) The complete, exact compiler error log along with the names and relevant lines of the files that failed to compile.

EXPLANATION

Precision self-healing relies on supplying deterministic stack traces alongside file names to avoid speculative iterations.

82. To prevent an automated coding agent from introducing breaking security vulnerabilities (e.g., hardcoded secrets, SQL injection), which architectural guardrail should be enforced?

- (A) Rely entirely on the model's systemic safety tuning to never write insecure code.
- (B) Integrate automated static application security testing (SAST) tools into the agent's post-generation execution chain.
- (C) Remove all database access blocks from the enterprise entirely.
- (D) Force the model to only use single-character variable names.

Answer: (B) Integrate automated static application security testing (SAST) tools into the agent's post-generation execution chain.

EXPLANATION

Automated post-generation SAST validation guarantees a reliable security guardrail that cannot be bypassed by system tuning bypasses.

- 83.** A team is using Claude to migrate a monolithic backend to microservices. The codebase is roughly 2 million lines of code. What is the most token-efficient way to handle this with Claude's context window?
- (A) Upload all 2 million lines of code in a single API request.
 - (B) Deconstruct the architecture into modular components, extract dependency graphs, and process individual modules sequentially.
 - (C) Convert the entire codebase into binary and feed it as a single chunk.
 - (D) Skip passing the source code and ask the model to generate the microservices based purely on a 1-sentence summary.

Answer: (B) Deconstruct the architecture into modular components, extract dependency graphs, and process individual modules sequentially.

EXPLANATION

Breaking giant text volumes down into modular graphs matches context scaling best practices and limits input cost bloat.

- 84.** Which environment variable is typically used to authenticate automated CLI tools like Claude Code against enterprise infrastructure securely?

- (A) ANTHROPIC_API_KEY
- (B) CLAUDE_PASSWORD
- (C) SYSTEM_ROOT_AUTH
- (D) GITHUB_USER_COOKIE

Answer: (A) ANTHROPIC_API_KEY

EXPLANATION

ANTHROPIC_API_KEY is the standard cross-platform credential environment variable for authenticating tool endpoints.

- 85.** When using an LLM to generate complex Docker configurations, what precaution must be taken regarding base images?

- (A) Base images should always be left blank for the model to invent.
- (B) Explicitly pin verified, secure, and minimal base images (like Alpine or distroless tags) in the system context/instructions rather than letting the model pick unverified tags.
- (C) Allow the model to pull random images from public unverified personal registries.
- (D) Avoid using base images entirely and build everything from scratch.

Answer: (B) Explicitly pin verified, secure, and minimal base images (like Alpine or distroless tags) in the system context/instructions rather than letting the model pick unverified tags.

EXPLANATION

Strict enterprise hardening mandates explicit pinning of secure base images inside context systems to prevent supply chain execution leaks.

- 86.** Your codebase tracking system needs to provide Claude with an understanding of project structure. What is the most lightweight, token-saving structural overview you can provide?

- (A) A full printout of every file's contents.
- (B) A clean directory tree diagram (e.g., generated via the tree command) showing paths and file names up to a reasonable depth.
- (C) A list of all file sizes in bytes.
- (D) A text file containing only the vowels found in the filenames.

Answer: (B) A clean directory tree diagram (e.g., generated via the tree command) showing paths and file names up to a reasonable depth.

EXPLANATION

Standard system directory map output summaries reduce structural over-indexing while supplying directory schema mapping.

- 87.** You want Claude 3.5 Sonnet to reliably output a complex nested JSON object matching an exact schema for an upstream API. Which technique is recommended?

- (A) Wrap the schema description in informal dialogue and set temperature to 1.0.
- (B) Provide a clear JSON schema definition using XML tags for structure, provide a concrete few-shot example of a valid JSON output, and set the temperature to 0.
- (C) Request the output in Markdown format first, then manually write regex to guess the keys.
- (D) Tell the model that if it fails to output perfect JSON, the connection will drop.

Answer: (B) Provide a clear JSON schema definition using XML tags for structure, provide a concrete few-shot example of a valid JSON output, and set the temperature to 0.

EXPLANATION

Combining explicit typing definitions, distinct XML tags, formatting anchors, and zero temperature maximizes structured compliance.

88. What is the structural role of XML tags (e.g., <instructions>, <context>, <examples>) when engineering prompts for Anthropic models?

- (A) They convert the prompt text directly into executable HTML web pages.
- (B) They serve as clear, semantic boundary markers that help Claude separate instructions from data inputs and context, reducing confusion.
- (C) They reduce the token count of the prompt by 50%.
- (D) They override the model's base safety alignment parameters.

Answer: (B) They serve as clear, semantic boundary markers that help Claude separate instructions from data inputs and context, reducing confusion.

EXPLANATION

Anthropic models are trained extensively on XML data structures to maintain strict semantic borders between code instructions and user parameters.

89. You are experiencing a high rate of hallucinations in a document summarization application. The model keeps adding facts not found in the original source text. How can you minimize this via prompt engineering?

- (A) Tell the model to think creatively and infer missing values.
- (B) Explicitly instruct the model to rely only on the provided text, state "if the information is not present, respond with 'Not found'", and use a clear thinking step before answering.
- (C) Delete the system prompt and put everything in the user prompt role.
- (D) Increase the temperature to 0.9.

Answer: (B) Explicitly instruct the model to rely only on the provided text, state "if the information is not present, respond with 'Not found'", and use a clear thinking step before answering.

EXPLANATION

Anchoring models directly to the provided text block while giving an explicit "escape phrase" for missing items minimizes hallucinations.

90. What is a key difference between the system prompt parameter and the user prompt parameter in the Anthropic Messages API?

- (A) The system prompt is meant for high-level instructions, behavioral constraints, and roles that persist across structural processing, while the user prompt represents the active query turn.
- (B) The system prompt is billed at 10x the rate of the user prompt.
- (C) The user prompt cannot accept text strings.
- (D) The system prompt can only be read by Claude 3 Opus.

Answer: (A) The system prompt is meant for high-level instructions, behavioral constraints, and roles that persist across structural processing, while the user prompt represents the active query turn.

EXPLANATION

The system parameter fixes static boundary conditions, persona constraints, and baseline rules across API context steps.

91. When providing few-shot examples in a prompt to guide Claude's formatting behavior, where should these examples ideally be placed according to Anthropic's documentation guidelines?

- (A) Interleaved randomly throughout the user's input text.
- (B) Inside distinct XML tags (e.g., <examples>) within the system prompt or early in the context, before the actual data to be processed.
- (C) At the very end of the prompt sequence, after the model's stop sequences.
- (D) Wrapped inside an image payload asset.

Answer: (B) Inside distinct XML tags (e.g., <examples>) within the system prompt or early in the context, before the actual data to be processed.

EXPLANATION

Structuring baseline context examples inside clean semantic brackets prevents confusion with input variables.

92. An engineer wants to force Claude to write an explanation before giving a final classification code (e.g., "Reasoning: ... Code: SUCCESS"). Why is this sequential ordering superior to returning the code first?

- (A) It generates fewer total tokens.
- (B) Generating reasoning first allows the model to compute intermediate thought pathways, improving the statistical likelihood of selecting the correct final classification token.
- (C) Returning the code first causes the API transaction to time out.
- (D) It allows the client application to skip parsing the code entirely.

Answer: (B) Generating reasoning first allows the model to compute intermediate thought pathways, improving the statistical likelihood of selecting the correct final classification token.

EXPLANATION

Forcing intermediate rationalization phases utilizes context calculations to increase final selection probability metrics.

93. Which parameter should be modified to make Claude's responses completely deterministic for recurring validation testing?

- (A) Set max_tokens to 1.
- (B) Set temperature to 0.0.
- (C) Set top_p to 1.0 and temperature to 1.0.
- (D) Add a random string to the system prompt.

Answer: (B) Set temperature to 0.0.

EXPLANATION

A zero-value temperature forces model token processing to follow deterministic pathways by disabling probabilistic variations.

94. You notice that Claude is failing to follow negative constraints (e.g., "Do not include any pleasantries or greetings"). What is an effective way to re-engineer this instruction for better compliance?

- (A) Write the negative constraint 50 times in uppercase letters.
- (B) Rephrase it as a positive constraint (e.g., "Start your response directly with the first data point. Omit all greetings, introductions, and polite transitions.") and enforce it with a few-shot example.
- (C) Increase the model's temperature to 1.0.
- (D) Switch to a model from a different vendor.

Answer: (B) Rephrase it as a positive constraint (e.g., "Start your response directly with the first data point. Omit all greetings, introductions, and polite transitions.") and enforce it with a few-shot example.

EXPLANATION

Positive prescriptive structural framing is processed more reliably than abstract negative constraints.

95. What happens if you specify a stop sequence of ["</json>"] in your API request to Claude?

- (A) The API will throw an invalid parameter error.
- (B) The model will immediately cease text generation as soon as it outputs the token string </json>, preventing further trailing text or tokens.
- (C) The model will skip generating JSON entirely.
- (D) The server will cache the prompt automatically.

Answer: (B) The model will immediately cease text generation as soon as it outputs the token string </json>, preventing further trailing text or tokens.

EXPLANATION

Stop sequences immediately interrupt API generations once the target character structure matches, optimizing token efficiency.

96. When asking Claude to process raw text data containing arbitrary symbols, brackets, and conversational text, how should you wrap that data to prevent prompt injection or parsing failures?

- (A) Enclose the raw data cleanly within descriptive XML tags like <user_data>...</user_data> and instruct the model to treat everything inside those tags as untrusted content.
- (B) Remove all spaces and punctuation from the raw text data.
- (C) URL-encode the text data into a single string line.
- (D) Paste it directly into the middle of your core instruction sentence.

Answer: (A) Enclose the raw data cleanly within descriptive XML tags like <user_data>...</user_data> and instruct the model to treat everything inside those tags as untrusted content.

EXPLANATION

Enclosing incoming string blocks in XML brackets creates clear data isolation lines to neutralize prompt injection vectors.

97. You want Claude to generate text in a highly specific, rare poetic style. Which approach gives the model the best guidance?

- (A) Give a 1-word instruction naming the style.
- (B) Provide explicit structural rules of the poetic form (syllables, rhyme scheme) along with multiple high-quality examples of that exact style inside XML tags.
- (C) Lower the max_tokens limit to 10 tokens.
- (D) Use Claude 3 Haiku without any instructions.

Answer: (B) Provide explicit structural rules of the poetic form (syllables, rhyme scheme) along with multiple high-quality examples of that exact style inside XML tags.

EXPLANATION

Structural rule declarations combined with illustrative style blueprints ensure maximum style adherence.

98. When designing prompts for high-volume enterprise production environments, why should you avoid overly verbose and repetitive phrasing in your instructions?

- (A) Claude cannot read prompts longer than 500 words.
- (B) Verbosity increases input token costs and input latency unnecessarily without guaranteeing a proportional increase in accuracy.
- (C) Repetitive phrasing causes the Anthropic API gateway to flag the request as spam.
- (D) Verbosity changes the output format to binary.

Answer: (B) Verbosity increases input token costs and input latency unnecessarily without guaranteeing a proportional increase in accuracy.

EXPLANATION

Redundant token noise inflates execution latency and overhead costs without driving equivalent accuracy returns.

99. An enterprise architecture requires Claude to communicate securely with an on-premise PostgreSQL database. Under the Model Context Protocol (MCP) framework, which component is directly responsible for establishing the database connection and running SQL queries?

- (A) The MCP Client
- (B) The MCP Server
- (C) The Anthropic Hosted API Cloud
- (D) The Claude Desktop Application Client

Answer: (B) The MCP Server

EXPLANATION

Within the Model Context Protocol architecture, the Server component directly manages downstream local data execution connections.

100When defining a tool's `input_schema` in the Claude API, what standard format must be used?

- (A) Protocol Buffers
- (B) XML Schema Definition (XSD)
- (C) JSON Schema (typically draft-07)
- (D) Raw YAML string blocks

Answer: (C) JSON Schema (typically draft-07)

EXPLANATION

Tool parameter descriptions are standard draft-07 JSON schemas parsing key-value object structures.

101What is the structural role of the `description` field within a tool definition payload submitted to the Claude API?

- (A) It is ignored by the model and used only for internal human developer logging.
- (B) It provides semantic context that tells Claude what the tool does, when it should be called, and what its parameters mean.
- (C) It defines the encryption key for the tool payload transmission.
- (D) It specifies the billing rate tier for the tool execution loop.

Answer: (B) It provides semantic context that tells Claude what the tool does, when it should be called, and what its parameters mean.

EXPLANATION

Tool descriptions function as critical semantic instructions used by the model's inner reasoning turns to select endpoints.

102Under the Model Context Protocol (MCP), what is the fundamental functional difference between a Resource and a Tool?

- (A) A Resource allows the model to perform read-only data inspection, while a Tool allows the model to execute actions and side effects.
- (B) A Resource can only be written in Python, while a Tool can only be written in TypeScript.
- (C) Tools are hosted on Anthropic's servers; Resources are hosted on the client machine.
- (D) There is no difference; they are synonymous terms in the specification.

Answer: (A) A Resource allows the model to perform read-only data inspection, while a Tool allows the model to execute actions and side effects.

EXPLANATION

Resources are read-only background context sources; Tools are active execution blocks engineered to generate side effects.

- 103**An application developer passes a tool definition called `get_weather` to Claude. Claude outputs a response containing a `tool_use` block. What action must the client application take next?
- (A) Do nothing; Claude executes the weather API internally on Anthropic's secure servers.
 - (B) Intercept the `tool_use` block, read the parameters, execute the local code or external weather API, and return the result to Claude in a `tool_result` message block.
 - (C) Throw a validation exception and terminate the session.
 - (D) Clear the chat history and restart the prompt turn.

Answer: (B) Intercept the `tool_use` block, read the parameters, execute the local code or external weather API, and return the result to Claude in a `tool_result` message block.

EXPLANATION

The client gateway abstracts tool execution, running local system code before feeding results back into the session history stack.

- 104**You are configuring an MCP integration for an engineering team using Claude Desktop. Where are the local MCP servers declared so the desktop client can discover and spin them up?
- (A) In a centralized system registry keys folder.
 - (B) Inside the `claude_desktop_config.json` configuration file under the `mcpServers` key block.
 - (C) In the environment path variable `$PATH`.
 - (D) By sending a POST request to `anthropic.com`.

Answer: (B) Inside the `claude_desktop_config.json` configuration file under the `mcpServers` key block.

EXPLANATION

Local client configurations specify MCP server paths, discovery arguments, and runtime variables inside the workspace profile.

- 105**An enterprise security policy states that Claude must never see unencrypted credit card tokens. However, Claude needs to trigger a payment processor tool. How should you design the tool schema parameters?
- (A) Have Claude accept the raw card details and trust the model to encrypt them.
 - (B) Have the tool accept a temporary, abstract `session_id` or `vault_token` that the backend system resolves internally without exposing raw credit card numbers to the LLM.
 - (C) Pass the credit card numbers inside an image asset payload.
 - (D) Disable the payment tool and ask the user to type the credit card number into a public forum.

Answer: (B) Have the tool accept a temporary, abstract `session_id` or `vault_token` that the backend system resolves internally without exposing raw credit card numbers to the LLM.

EXPLANATION

Abstract token routing structures prevent toxic or restricted data items from entering the transformer history layers.

- 106**When configuring the `tool_choice` parameter in the Anthropic Messages API, setting it to `{"type": "any"}` instructs the model to do what?
- (A) Refuse to use any tools and only output a conversational text response.
 - (B) Force the model to select and call at least one of the provided tools, bypassing conversational response text if necessary.
 - (C) Pick a tool from a completely random third-party public directory.
 - (D) Let the model choose whether to use a tool or return text freely.

Answer: (B) Force the model to select and call at least one of the provided tools, bypassing conversational response text if necessary.

EXPLANATION

Setting any-type tool choice locks the model into an immediate tool allocation mode, forcing structural programmatic turns.

- 107**An MCP server communicates with an MCP client (like Claude Desktop) over what standard transport protocol for local processes?
- (A) WebSockets over TLS
 - (B) Standard Input/Output (stdio) using JSON-RPC 2.0 messages
 - (C) gRPC over HTTP/2
 - (D) FTP raw data streams

Answer: (B) Standard Input/Output (stdio) using JSON-RPC 2.0 messages

EXPLANATION

Local host-to-server MCP configurations operate over JSON-RPC 2.0 communication protocols via standard I/O pipes.

108What is an MCP Prompt primitive as defined by the Model Context Protocol specification?

- (A) An automated system that deletes user history logs.
- (B) A server-exposed slice of reusable prompt templates and workflows that the client can discover and present to the user or model.
- (C) A hardware acceleration chip used to train Claude models.
- (D) A mandatory security password required to log into Claude Code.

Answer: (B) A server-exposed slice of reusable prompt templates and workflows that the client can discover and present to the user or model.

EXPLANATION

Prompt primitives expose centralized templates and conversational workflows directly out of the server layers.

109If a tool parameter is critical for execution (e.g., `account_id` for a banking lookup), how should this be enforced in the tool definition payload?

- (A) Mention it casually in the tool's description text.
- (B) Include the parameter name string within the required array of the tool's JSON schema block.
- (C) Create a separate tool called `require_account_id`.
- (D) Set the model's temperature to exactly 0.5.

Answer: (B) Include the parameter name string within the required array of the tool's JSON schema block.

EXPLANATION

Marking critical schema components inside the JSON required list explicitly enforces structural runtime compliance.

110An enterprise legal application processes 200-page contracts repeatedly throughout the day. To optimize API latency and reduce input token expenses, the architect should utilize which Anthropic feature?

- (A) Token compression arrays
- (B) Prompt Caching (context caching)
- (C) Local context window slicing
- (D) Dynamic model compilation

Answer: (B) Prompt Caching (context caching)

EXPLANATION

Context caching patterns allow massive reference datasets to be retained in server memory pools to lower cost parameters.

111What is the structural requirement for an exact cache hit when using Anthropic's Prompt Caching mechanism?

- (A) The system prompt must change completely on every single API request turn.
- (B) The prompt prefix (from the beginning of the prompt up to the cache breakpoint marker) must be structurally and textually identical to a previously cached sequence.
- (C) The user must be connecting from the exact same IP address.
- (D) The `max_tokens` parameter must be set to an odd number.

Answer: (B) The prompt prefix (from the beginning of the prompt up to the cache breakpoint marker) must be structurally and textually identical to a previously cached sequence.

EXPLANATION

Cache matches demand absolute binary character and structure identity starting from the absolute beginning of the string layout.

112You are implementing prompt caching for a customer service chatbot. You place the `current_timestamp` and `user_session_id` at the very beginning of the system prompt, followed by a 40,000-token corporate policy knowledge base document. What is the impact on cache performance?

- (A) Cache hit rate will be 100% because the massive document is included.
- (B) Cache hit rate will drop to 0% because the dynamic variables at the beginning change every time, invalidating the exact prefix match required for the subsequent static text.
- (C) The API will automatically move the dynamic variables to the end of the prompt.
- (D) The cost will be reduced by 90% regardless of the order.

Answer: (B) Cache hit rate will drop to 0% because the dynamic variables at the beginning change every time, invalidating the exact prefix match required for the subsequent static text.

EXPLANATION

Prepending variable metadata strings early inside the input structure shifts alignment positions, invalidating downstream prefix matching.

113What is the minimum token threshold required to trigger or construct a cache milestone for Claude 3.5 Sonnet prompts?

- (A) 10 tokens
- (B) 1,024 tokens
- (C) 100,000 tokens
- (D) 50,000 tokens

Answer: (B) 1,024 tokens

EXPLANATION

The baseline entry milestone for caching mechanisms requires an initial character text depth of 1,024 prompt tokens.

114An application needs to summarize a high-volume flow of incoming medical logs while ensuring the highest level of data privacy and reliability. Which deployment choice aligns with strict enterprise governance?

- (A) Sending the logs via a public unencrypted third-party proxy website.
- (B) Using Anthropic's API endpoints configured with zero data retention commitments or enterprise cloud-provider gateways (e.g., AWS Bedrock or Google Cloud Vertex AI) that guarantee data isolation.
- (C) Storing the logs in a public GitHub repository for easy API fetching.
- (D) Disabling all API security tokens to speed up delivery.

Answer: (B) Using Anthropic's API endpoints configured with zero data retention commitments or enterprise cloud-provider gateways (e.g., AWS Bedrock or Google Cloud Vertex AI) that guarantee data isolation.

EXPLANATION

High-compliance ecosystems rely on private cloud infrastructure tenancy models to manage pipeline data residency layers.

115When designing a system to handle token limit exceptions defensively, what is the best metric to monitor on the client application side from the API response?

- (A) The response latency time in milliseconds.
- (B) The usage metadata block, specifically input_tokens and output_tokens.
- (C) The server IP address string.
- (D) The version code of the operating system.

Answer: (B) The usage metadata block, specifically input_tokens and output_tokens.

EXPLANATION

API token logging records usage dimensions directly inside returned transport metadata blocks.

116Your application experiences a sudden surge in traffic, resulting in HTTP 429 (Too Many Requests) rate limit errors from the Anthropic API. What is the correct architectural pattern to manage this gracefully?

- (A) Immediately retry the request in an unconstrained loop as fast as possible.
- (B) Implement an exponential backoff retry strategy with jitter in the client gateway orchestrator.
- (C) Crash the application server and force a hard reboot.
- (D) Lower the API key security permission level.

Answer: (B) Implement an exponential backoff retry strategy with jitter in the client gateway orchestrator.

EXPLANATION

Rate limit exceptions demand retry pacing protocols built on randomized exponential delays to protect API pathways.

117Why should system architectures decouple long-term enterprise knowledge (e.g., 500 product manuals) into a Retrieval-Augmented Generation (RAG) vector database rather than feeding everything into Claude's massive context window every single turn?

- (A) Claude cannot read text related to product manuals.
- (B) It optimizes both cost and operational latency by retrieving only the most relevant text chunks for a query, avoiding wasting tokens on irrelevant context.
- (C) Vector databases automatically correct syntax errors in Claude's output.
- (D) Decoding speed is completely independent of prompt size.

Answer: (B) It optimizes both cost and operational latency by retrieving only the most relevant text chunks for a query, avoiding wasting tokens on irrelevant context.

EXPLANATION

RAG pipelines eliminate extensive token footprint overhead by routing only highly relevant historical chunks into the system window.

118 When utilizing Claude 3.5 Sonnet for critical automated assessments, what is the primary reason for configuring a dual-region or fallback endpoint architecture (e.g., primary on AWS Bedrock, fallback on Anthropic API direct)?

- (A) It makes the model execute responses twice as fast under normal loads.
- (B) It ensures high availability and business continuity in the event of regional cloud outages or strict API rate limit exhaustion.
- (C) It alters the underlying weights of the model dynamically.
- (D) It is a mandatory requirement enforced by the JSON schema standard.

Answer: (B) It ensures high availability and business continuity in the event of regional cloud outages or strict API rate limit exhaustion.

EXPLANATION

Cross-regional high-availability architectures preserve runtime continuity against regional infrastructure link downlines.

Claude CCAF Practice Mock 3 (Exam Heist Scenario Simulation - 53Q)

Mock Tests - 53 questions

119 After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

- (A) The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.
- (B) The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.
- (C) The subagents need to share a single API connection to enable automatic context sharing between invocations.
- (D) The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

Answer: (D) The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

EXPLANATION

Correct. The synthesis agent can only act on the information provided in its prompt. If prior outputs are not passed, it will report missing research findings.

Why the other options are wrong

- **A.** Incorrect. Agents do not require direct access to each other's histories. Proper orchestration passes outputs explicitly via prompts.
- **B.** Incorrect. If this were the issue, the agent would receive truncated data, not no data at all. The error indicates missing inputs entirely.
- **C.** Incorrect. Agent communication does not depend on shared API connections. Context must be explicitly passed by the coordinator.

120 When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2022: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

- (A) Require subagents to include publication or data collection dates in their structured outputs.
- (B) Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.
- (C) Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.
- (D) Configure the web search agent to only return results from the past 6 months.

Answer: (A) Require subagents to include publication or data collection dates in their structured outputs.

EXPLANATION

Correct. Providing timestamps allows the synthesis agent to understand that the figures refer to different points in time, enabling it to interpret the data as a trend (growth) rather than a contradiction.

Why the other options are wrong

- **B.** Incorrect. This approach hides useful context and does not help the agent understand relationships between data points over time.
- **C.** Incorrect. Discarding older data removes valuable historical insight and prevents trend analysis.
- **D.** Incorrect. Limiting recency reduces context and does not address the core issue of interpreting time-based differences.

121 Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps - for instance, noting "the retrieved sources discuss API authentication but lack details on token refresh patterns" - but under the current strict pipeline, this insight isn't actionable since search has already completed. What is the most effective architectural change?

- (A) Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.
- (B) Add a research planning agent before the search phase that decomposes topics into specific sub-questions.
- (C) Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.
- (D) Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.

Answer: (A) Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.

EXPLANATION

This introduces a dynamic, agentic loop (or reflection pattern) into the workflow. Instead of a rigid, linear pipeline where steps cannot be retraced, the system can now adapt based on what it discovers. The analysis agent is the one actively reading the text and identifying exactly what is missing (e.g. "missing token refresh patterns"). By reporting these specific gaps back to the coordinator, the coordinator can intelligently route the workflow back to the search agent with a highly targeted query, then pass the new findings back to the analysis agent to close the loop.

Why the other options are wrong

- **B.** (no explanation in source - the source PDF's explanation section for this question ends early)
- **C.** (no explanation in source - the source PDF's explanation section for this question ends early)
- **D.** (no explanation in source - the source PDF's explanation section for this question ends early)

122. Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach best balances information fidelity with context efficiency when restoring agent state?

- (A) Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.
- (B) Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.
- (C) Have each agent maintain its own persistent state file and reload it independently at the start of each session.
- (D) Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.

Answer: (A) Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

EXPLANATION

Correct. This provides high information fidelity (structured, complete outputs) while maintaining context efficiency (only relevant pieces are re-injected into prompts). The coordinator remains in control of what each agent needs, avoiding unnecessary bloat and duplication.

Why the other options are wrong

- **B.** Incorrect. Conversation logs are often verbose and unstructured, leading to context overload and inefficient prompt usage without guaranteed clarity.
- **C.** Incorrect. This decentralizes control and can lead to inconsistencies and coordination issues, especially when agents need shared or aligned context.
- **D.** Incorrect. Vector stores are useful for retrieval, but they introduce probabilistic recall and may miss or distort critical structured state, reducing fidelity during recovery.

123. The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

- (A) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.
- (B) Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.
- (C) Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.
- (D) Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

Answer: (A) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.

EXPLANATION

Correct. This introduces an iterative feedback loop, where identified gaps are actively addressed. The coordinator maintains control and ensures completeness before final report generation.

Why the other options are wrong

- **B.** Incorrect. Broader queries may help coverage but are inefficient and still won't guarantee that specific gaps discovered later are filled.
- **C.** Incorrect. This improves transparency but does not solve the completeness problem.
- **D.** Incorrect. This breaks separation of concerns and reduces system control. The coordinator should manage task delegation, not the synthesis agent.

124 When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- (A) Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations.
- (B) Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents.
- (C) Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity.
- (D) Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis.

Answer: (D) Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis.

EXPLANATION

Correct. This enables parallel processing to reduce latency while keeping orchestration centralized. The coordinator retains full visibility, making monitoring and debugging easier.

Why the other options are wrong

- **A.** Incorrect. This decentralizes orchestration and makes the system harder to monitor and debug. The coordinator loses visibility into dynamically spawned agents.
- **B.** Incorrect. While this improves scalability, it introduces infrastructure complexity and reduces transparency for debugging at the coordinator level.
- **C.** Incorrect. This further complicates the architecture and makes tracing execution paths difficult, reducing observability and control.

125 Monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent - neither requires the other's output. How should you modify the system to run these subagents concurrently?

- (A) Switch both subagents to use a Haiku tier model to reduce their individual execution time.
- (B) Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator subagent pair, then aggregates results.
- (C) Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same time.
- (D) Structure the coordinator to emit both Task tool calls in a single response message rather than across separate conversation turns.

Answer: (D) Structure the coordinator to emit both Task tool calls in a single response message rather than across separate conversation turns.

EXPLANATION

Correct. Issuing both tool calls in one response enables true parallel execution, since the system can run them concurrently instead of waiting for one to finish before starting the other.

Why the other options are wrong

- **A.** Incorrect. This may reduce latency per task, but it does not address the core issue of sequential execution vs. parallelism.
- **B.** Incorrect. This overcomplicates the architecture and duplicates coordinators unnecessarily instead of fixing concurrency within the existing flow.
- **C.** Incorrect. Instructions alone are not reliable for enforcing concurrency. Execution behavior depends on how tool calls are structured, not just prompt wording.

126 Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), while other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate \$50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates \$35B (\$7B, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the market may be \$35B-\$50B depending on factors." What systematic approach best addresses this?

(A) Configure subagents to only report findings meeting a high confidence threshold, filtering uncertain information before it reaches the coordinator.

(B) Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.

(C) Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterization and methodological context.

(D) Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calculated reliability scores to produce a statistically grounded synthesis.

Answer: (C) Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterization and methodological context.

EXPLANATION

Correct. This directly addresses inconsistent handling of uncertainty by making it explicit and structured, allowing users to understand both consensus and disagreement without losing context.

Why the other options are wrong

- **A.** Incorrect. This suppresses potentially valuable but uncertain insights and introduces bias by hiding ambiguity rather than managing it.
- **B.** Incorrect. While useful for validation, this approach still filters out uncertainty instead of representing it, and may discard novel or emerging insights.
- **D.** Incorrect. This introduces artificial precision and may oversimplify complex, qualitative uncertainty, potentially misleading users.

127 A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with margins and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technologies. A synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

(A) Update the synthesis agent to render each content type appropriately - financial data as tables, news as prose, and technical lists as structured points.

(B) Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation (such as Markdown) to facilitate more flexible rendering.

(C) Standardize all subagent outputs to JSON with fields for every data type to ensure programmatic consistency across the pipeline.

(D) Standardize all subagent outputs to prose summaries with a uniform character to maintain a consistent executive voice regardless of the source material.

Answer: (A) Update the synthesis agent to render each content type appropriately - financial data as tables, news as prose, and technical lists as structured points.

EXPLANATION

Correct. This preserves the natural structure and strengths of each data type, improving clarity, readability, and usefulness of the final briefing.

Why the other options are wrong

- **B.** Incorrect. While helpful for consistency, this does not guarantee appropriate presentation of different content types and may still lead to generic formatting.
- **C.** Incorrect. This improves structure but shifts complexity to the synthesis stage and does not inherently improve human-readable output quality.
- **D.** Incorrect. This sacrifices important structure (like tables and lists), reducing clarity and effectiveness for data-heavy content.

128The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate - it generates messages like "I'll ask the web search agent to find sources on this topic" - but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

(A) Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.

(B) The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

(C) The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.

(D) The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.

Answer: (C) The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.

EXPLANATION

Correct. The coordinator can plan and describe delegation, but without the Task tool enabled, it cannot actually execute subagent calls - resulting in no errors but no execution.

Why the other options are wrong

- **A.** Incorrect. Even with context isolation, subagents would still be invoked - the issue here is that no invocation happens at all, not that context is missing.

- **B.** Incorrect. Token limits might truncate responses, but this would typically produce malformed outputs or errors - not silent absence of any tool calls.

- **D.** Incorrect. While listing agents can help, the model already demonstrates awareness ("I'll ask the web search agent..."). The problem is execution capability, not awareness.

129In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

(A) Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.

(B) Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations.

(C) Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

(D) Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

Answer: (D) Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

EXPLANATION

Correct. This ensures end-to-end attribution fidelity by keeping claim-to-source relationships explicit and structured throughout the pipeline, preventing loss during synthesis.

Why the other options are wrong

- **A.** Incorrect. This relies on post-hoc inference, which is error-prone and can misattribute claims due to semantic ambiguity.

- **B.** Incorrect. This is a fragile, text-based workaround that can break during transformations and doesn't scale well.

- **C.** Incorrect. This adds unnecessary complexity and still depends on indirect reconstruction rather than preserving attribution explicitly.

130After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

(A) Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks.

(B) Include the complete findings from both subagents directly in the synthesis subagent's prompt.

(C) Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results.

(D) Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator.

Answer: (C) Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results.

EXPLANATION

Correct. This is the most scalable and production-ready approach. It preserves information fidelity while avoiding context bloat, allowing the synthesis agent to retrieve exactly what it needs.

Why the other options are wrong

- **A.** Incorrect. This introduces unnecessary coupling and complexity. Subagents shouldn't need to actively fetch data from others.
- **B.** Incorrect. While simple, this approach does not scale well for large outputs and can exceed context limits, reducing efficiency.
- **D.** Incorrect. There is no automatic context inheritance - without explicit data access, the synthesis agent cannot function properly.

131. The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information flow between these two specialized subagents?

- (A) The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.
- (B) The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.
- (C) The web search agent directly invokes the document analysis agent, using the discovered sources as parameters.
- (D) Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.

Answer: (A) The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

EXPLANATION

Correct. This follows the standard orchestration pattern where the coordinator manages all data flow, explicitly passing outputs between subagents.

Why the other options are wrong

- **B.** Incorrect. This introduces unnecessary infrastructure complexity and is not the typical agent orchestration model.
- **C.** Incorrect. Subagents should not invoke each other directly; this breaks centralized control and observability.
- **D.** Incorrect. While possible in advanced systems, this is not the standard or simplest approach; it adds complexity without clear necessity in typical pipelines.

132. After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

- (A) Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.
- (B) Pass the full accumulated context from all prior agents.
- (C) Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.
- (D) Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.

Answer: (C) Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.

EXPLANATION

Correct. This provides the best balance of completeness and efficiency - retaining precise attribution while keeping context size manageable.

Why the other options are wrong

- **A.** Incorrect. This relies on post-hoc reconstruction, which is error-prone and can lead to incorrect or missing citations.
- **B.** Incorrect. This ensures completeness but is highly inefficient (120K+ tokens) and risks exceeding context limits.
- **D.** Incorrect. This loses granularity and makes precise citation mapping difficult, reducing attribution fidelity.

- 133.** Your `search_products` tool queries an external catalog API that returns paginated results (50 items per request). Production logs show queries frequently match 200+ products, and the design that auto-fetches all pages causes 15-20 second delays. How should you redesign the pagination handling?
- (A) Create separate `search_products` and `fetch_more_results` tools for pagination.
 - (B) Implement server-side relevance ranking and return only the top 50 most relevant items.
 - (C) Add a `max_pages` parameter (default: 2) that controls how many pages are fetched internally.
 - (D) Return the first page with total match count and cursor for additional pages.

Answer: (D) Return the first page with total match count and cursor for additional pages.

EXPLANATION

Correct. This enables lazy loading and explicit control, allowing the agent to fetch more results only when needed - balancing performance and completeness.

Why the other options are wrong

- **A.** Incorrect. This exposes pagination mechanics to the agent, increasing complexity and coupling tool usage with control flow.
- **B.** Incorrect. While this reduces latency, it removes access to the full result set, limiting flexibility when more results are actually needed.
- **C.** Incorrect. This is an improvement over fetching everything, but it still hides pagination control inside the tool and may fetch unnecessary data.

- 134.** Your `search_flights` tool calls an external airline API that occasionally returns a 503 Service Unavailable error. What is the most effective way to handle this error in your tool implementation?

- (A) Return an empty flight list as if the search succeeded but found no matching flights.
- (B) Log the error internally and return an empty response, letting the model continue without the flight data.
- (C) Return an error message in the tool result explaining the service is temporarily unavailable.
- (D) Automatically retry the request up to five times with exponential backoff before returning results to the agent.

Answer: (D) Automatically retry the request up to five times with exponential backoff before returning results to the agent.

EXPLANATION

Correct. This is the most effective approach - handles transient failures gracefully, improves reliability, and only surfaces errors if retries fail.

Why the other options are wrong

- **A.** Incorrect. This hides the failure and misleads the system into thinking no flights exist, which can lead to incorrect conclusions.
- **B.** Incorrect. This still suppresses the failure signal, preventing the agent from taking corrective action.
- **C.** Incorrect. While transparent, this alone doesn't attempt recovery and may degrade user experience unnecessarily.

- 135.** Your MCP server implements a `check_availability` tool that queries an external calendar API. During testing, you encounter three error conditions: (1) the tool is called with a malformed request, missing the required `user_email` parameter, (2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system, (3) the calendar API returns a 503 because the service is temporarily unavailable. How should each error be reported according to MCP's error handling design?

- (A) Report all three as tool results with `isError: true`.
- (B) Report errors 1 and 2 as JSON-RPC protocol errors, report error 3 as a tool result with `isError: true`.
- (C) Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with `isError: true`.
- (D) Report all three as JSON-RPC protocol errors.

Answer: (C) Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with `isError: true`.

EXPLANATION

Correct. Error 1 (malformed request) -> JSON-RPC protocol error (invalid input). Error 2 (user not found) -> tool result with `isError: true` (valid execution, meaningful failure). Error 3 (service unavailable) -> tool result with `isError: true` (transient external failure).

Why the other options are wrong

- **A.** Incorrect. Malformed requests (error 1) are protocol-level issues, not tool execution results, so they should not be reported this way.
- **B.** Incorrect. A 404 (error 2) is a valid tool execution outcome (the user doesn't exist), not a protocol error.
- **D.** Incorrect. Only malformed requests should be protocol errors; external API responses are tool-level outcomes, not protocol failures.

136.Your `search_documents(query)` tool returns results as "Found 3 documents: Q2 Budget Proposal, Q2 Budget Forecast, Annual Review". You want the agent to be able to act on a specific document in follow-up steps. What return format would best enable these multi-step workflows?

- (A) URLs that users can click to open the document in their browser.
- (B) Structured data containing document IDs and metadata for each result.
- (C) A JSON array of document titles extracted from the search results.
- (D) More detailed human-readable descriptions including the size and authors.

Answer: (B) Structured data containing document IDs and metadata for each result.

EXPLANATION

Correct. This enables the agent to programmatically reference specific documents (via IDs) across multiple steps, making workflows like follow-up queries or document retrieval precise and reliable.

Why the other options are wrong

- **A.** Incorrect. URLs are useful for users, but not ideal for agents performing multi-step workflows that require reliable referencing and further operations.
- **C.** Incorrect. Titles alone are ambiguous and not stable identifiers, making it difficult for agents to reliably act on specific documents.
- **D.** Incorrect. Helpful for users, but still unstructured and not suitable for precise multi-step agent operations.

137.Your agent has access to 50+ specialized API connectors for different external services. As the connector library grew, tool selection accuracy dropped to 58%. You design a `search_connectors(description)` tool that finds matching connectors, but in testing agents frequently skip searching and call connectors directly (often incorrectly), or search and select wrong connectors from the filtered results. How should you design the tool composition pattern to address both issues?

- (A) Design connectors with built-in compatibility validation that return descriptive errors for mismatched requests.
- (B) Design a `find_and_execute(description, params)` composite tool that searches and immediately executes the best matching connector.
- (C) Enhance all connector descriptions with detailed usage samples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow.
- (D) Design `search_connectors` to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered.

Answer: (D) Design `search_connectors` to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered.

EXPLANATION

Correct. This enforces the search-first pattern by limiting available tools initially and reducing the decision space, improving both discovery and correct selection.

Why the other options are wrong

- **A.** Incorrect. This helps with error handling after a wrong choice is made, but does not improve initial tool selection accuracy.
- **B.** Incorrect. This removes transparency and control, making debugging harder and preventing the agent from reasoning about tool choice.
- **C.** Incorrect. While helpful, this still relies on the agent to follow instructions and does not enforce correct behavior, especially at scale with 50+ tools.

138.Your `publish_article` tool calls an external CMS API that occasionally returns transient errors (network timeouts, 503s) and non-transient errors (403 permission denied, 422 validation failure). Currently, every error is returned directly to the agent, which leads to the agent retrying non-transient errors and wasting turns on failures that will never succeed. How should you partition error-handling responsibility between the tool implementation and the agent?

- (A) Handle all errors inside the tool: implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.
- (B) Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation failures) to the agent with descriptive messages so it can take corrective action.
- (C) Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times - keeping the tool implementation stateless and simple.
- (D) Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable - try again later" message, shielding the agent from error complexity.

Answer: (B) Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation failures) to the agent with descriptive messages so it can take corrective action.

EXPLANATION

Correct. This cleanly separates responsibility: the tool handles recoverable/transient issues automatically, and the agent receives actionable errors it can fix (permissions, input validation).

Why the other options are wrong

- **A.** Incorrect. This wastes time retrying non-transient errors (e.g., 403, 422) that will never succeed and hides useful feedback from the agent.
- **C.** Incorrect. This pushes retry logic to the agent, leading to inefficient behavior and wasted turns.
- **D.** Incorrect. This removes critical detail, preventing the agent from taking corrective actions when possible.

139. Your `remove_team_member` tool uses a `dry_run`: boolean parameter for previewing impacts before execution. Production monitoring shows the agent bypasses the preview step in 15% of calls by calling with `dry_run=false` directly. You need to ensure every removal is preceded by a preview that the user explicitly confirms. What is the most reliable approach?

- (A) Add server-side validation that permits `dry_run=false` only when a `dry_run=true` call with identical parameters occurred within the past 60 seconds.
- (B) Replace with two tools: `preview_remove_member` returns impact details and a single-use confirmation token; `execute_remove_member` requires that token, binding execution to the specific previewed action.
- (C) Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools.
- (D) Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with `dry_run=true` first and wait for user confirmation before calling with `dry_run=false`.

Answer: (B) Replace with two tools: `preview_remove_member` returns impact details and a single-use confirmation token; `execute_remove_member` requires that token, binding execution to the specific previewed action.

EXPLANATION

Correct. This enforces the correct workflow at the system level by requiring a valid preview step and tying execution to an explicit confirmation, making bypass impossible.

Why the other options are wrong

- **A.** Incorrect. This approach is brittle because it depends on timing and does not guarantee that the user actually reviewed or confirmed the preview.
- **C.** Incorrect. This depends on orchestration behavior and is not strictly enforced, so it can still be bypassed or misconfigured.
- **D.** Incorrect. Instruction-based approaches are not reliable for enforcement, as demonstrated by the existing bypass rate.

140. Your expense reimbursement agent processes employee requests using a `process_reimbursement` tool. Company policy requires that reimbursements above \$500 must be approved before funds are disbursed. The agent handles hundreds of requests daily, and you need the threshold enforcement to be tamper-proof regardless of how the agent is prompted. What ensures the \$500 approval threshold cannot be bypassed?

- (A) The `process_reimbursement` tool accepts an `approved_by_manager` parameter. The system prompt instructs the agent to only set this to true after confirming that a manager approved the request. A nightly audit script reviews all reimbursements where `approved_by_manager` was set to true.
- (B) Provide two tools: `auto_reimburse` (hard-coded limit of \$500) and `manager_approval`. Include detailed system prompt instructions telling the agent to check the amount and use the appropriate tool. Add a `PostToolUse` hook that logs which tool was called for auditing.
- (C) The `process_reimbursement` tool accepts amount and details, and internally enforces the threshold; amounts <\$500 are auto-disbursed and the tool returns a success confirmation. Amounts >\$500 cause the tool to create a pending approval request and return a status indicating manager review is pending.
- (D) Implement the threshold check in a `PreToolUse` hook that inspects the amount parameter before `process_reimbursement` executes. If the amount exceeds \$500, the hook modifies the context to add a `requires_approval: true` flag, which the tool checks before disbursing.

Answer: (C) The `process_reimbursement` tool accepts amount and details, and internally enforces the threshold; amounts <\$500 are auto-disbursed and the tool returns a success confirmation. Amounts >\$500 cause the tool to create a pending approval request and return a status indicating manager review is pending.

EXPLANATION

Correct. This enforces the rule inside the tool itself, making it impossible to bypass regardless of how the agent is prompted.

Why the other options are wrong

- **A.** Incorrect. This relies on the agent following instructions and post-hoc auditing, which is not tamper-proof and allows bypass at execution time.
- **B.** Incorrect. Again depends on agent behavior and correct tool selection. Logging helps auditing but does not prevent misuse.
- **D.** Incorrect. `PreToolUse` hooks can be bypassed or misconfigured and still rely on downstream logic. Enforcement should reside directly within the tool for full reliability.

141.Your order management system requires tools for three distinct operations: issuing refunds (requires amount and reason), canceling orders (requires reason), and requesting reshipments (requires shipping address). Each operation shares an `order_id` parameter but has different additional requirements. You notice during testing that with your current unified tool design, the agent frequently omits required parameters or includes irrelevant ones. What design change will most effectively improve parameter accuracy?

- (A) Split into three separate tools, each defining only the parameters required for that specific operation.
- (B) Keep one unified tool with all parameters marked optional, but add few-shot examples in the system prompt showing correct parameter combinations for each operation.
- (C) Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like amount are required only when the operation type is "refund".
- (D) Keep one unified tool with a nested operation object parameter whose internal structure varies by operation type, documented in the tool description.

Answer: (A) Split into three separate tools, each defining only the parameters required for that specific operation.

EXPLANATION

Correct. This reduces ambiguity and ensures the agent only sees relevant parameters per operation, leading to much higher accuracy.

Why the other options are wrong

- **B.** Incorrect. Examples help, but the schema remains ambiguous, so errors will still occur.
- **C.** Incorrect. While technically valid, this increases complexity and is less reliable than simply separating tools.
- **D.** Incorrect. This adds complexity and cognitive load, making it harder for the agent to consistently provide correct parameters.

142.Your `portfolio_value` tool returns the total value of a user's investment portfolio. You're deciding between returning a structured JSON object with explicit fields versus returning information as a formatted text string. What is the primary advantage of using structured output with defined fields?

- (A) Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs.
- (B) The agent can reliably extract specific values without parsing free-form text, reducing errors in subsequent operations.
- (C) Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values.
- (D) JSON schemas automatically validate that the underlying API returned correct data before the agent processes it.

Answer: (B) The agent can reliably extract specific values without parsing free-form text, reducing errors in subsequent operations.

EXPLANATION

Correct. Structured output provides clear, predictable fields, making it easy for the agent to use the data accurately in downstream steps.

Why the other options are wrong

- **A.** Incorrect. Token usage depends on the content; JSON is not inherently more compact than text and may sometimes use more tokens.
- **C.** Incorrect. The model is still probabilistic; JSON improves structure, but not deterministic processing.
- **D.** Incorrect. Schemas define structure, but they do not guarantee correctness of the actual data returned by the API.

143.Your scheduling agent uses `get_available_slots(date, provider_id)` to retrieve open appointment times, then `book_appointment(provider_id, slot_time, patient_id)` to reserve a slot. Metrics show that 15% of booking attempts fail with "slot no longer available" because another user booked the slot between the availability check and the booking call. How should you redesign these tools?

- (A) Modify `book_appointment` to return detailed failure information including currently available alternative slots when the requested slot is unavailable, enabling the agent to retry with a different time.
- (B) Keep both tools but add retry logic to the agent's system prompt, instructing it to call `get_available_slots` again and select a different time if booking fails.
- (C) Add a `hold_slot(provider_id, slot_time)` tool that creates a 60 second temporary reservation, requiring the agent to call it between checking availability and booking.
- (D) Combine both tools into a single `find_and_book_appointment` that atomically checks availability and books, returning either the confirmed booking or available alternatives.

Answer: (D) Combine both tools into a single `find_and_book_appointment` that atomically checks availability and books, returning either the confirmed booking or available alternatives.

EXPLANATION

Correct. This eliminates the race condition by making the operation atomic, ensuring consistency and reliability.

Why the other options are wrong

- **A.** Incorrect. This improves recovery but does not fix the race condition between availability check and booking.

- **B.** Incorrect. This still suffers from the same race condition and relies on agent behavior rather than fixing the underlying issue.
- **C.** Incorrect. This reduces the issue but introduces additional complexity and still requires multiple steps that can fail.

144. Your agent has a `log_workout` tool that accepts `exercise_type` (string), `value` (number), and `measurement` (string). Production monitoring shows the agent frequently passes mismatched combinations - using `measurement: "reps"` for cardio exercises like running, or `measurement: "miles"` for strength exercises like bench press. Your exercises naturally divide into two categories: cardio (measured in time or distance) and strength (measured in reps and sets). 23% of tool calls have invalid combinations. What approach would most effectively reduce these errors?

- (A) Implement server-side validation returning descriptive errors for invalid combinations, allowing the agent to retry with corrections.
- (B) Add enum constraints on `measurement` limiting values to "minutes", "miles", "reps", or "sets" to prevent arbitrary measurement strings.
- (C) Add explicit examples to the tool description showing valid combinations (e.g., "For running: use minutes or miles. For push-ups: use reps") with constraints for each exercise category.
- (D) Split into `log_cardio_workout` (with `duration_minutes` or `distance_miles` parameters) and `log_strength_workout` (with `reps` and `sets` parameters).

Answer: (D) Split into `log_cardio_workout` (with `duration_minutes` or `distance_miles` parameters) and `log_strength_workout` (with `reps` and `sets` parameters).

EXPLANATION

Correct. This enforces correctness at the schema level by eliminating invalid parameter combinations entirely, significantly reducing errors.

Why the other options are wrong

- **A.** Incorrect. This catches errors after they occur but does not prevent them, leading to wasted turns and inefficiency.
- **B.** Incorrect. This restricts values but does not prevent invalid combinations (e.g., still allows "miles" for bench press).
- **C.** Incorrect. Helpful guidance, but not enforceable - agents can still make mistakes.

145. Your MCP server includes `archive_file(file_id)` and `delete_file(file_id)` tools. Production logs show the agent calls `delete_file` when users ask to "remove old backups," but policy requires archiving backup files. Both tools currently have minimal descriptions: "Archives a file" and "Deletes a file." Which change most directly improves tool selection?

- (A) Add a confirmation step that requires users to type "CONFIRM DELETE" before `delete_file` executes.
- (B) Implement server-side validation that rejects `delete_file` calls for files tagged as backups, returning an error message suggesting `archive_file`.
- (C) Expand tool descriptions to clarify use cases, adding guidance like "Do not use for backup files" to `delete_file`.
- (D) Add few-shot examples to the system prompt demonstrating that requests involving "backup" or "old" should use `archive_file`.

Answer: (C) Expand tool descriptions to clarify use cases, adding guidance like "Do not use for backup files" to `delete_file`.

EXPLANATION

Correct. Clear, specific descriptions directly influence the agent's tool selection reasoning, making it less likely to choose the wrong tool.

Why the other options are wrong

- **A.** Incorrect. This prevents accidental execution but does not improve the agent's tool selection decision.
- **B.** Incorrect. This enforces policy after the wrong choice is made but does not directly improve initial selection.
- **D.** Incorrect. Helpful, but less direct and less reliable than improving the tool descriptions themselves.

146. Your CRM agent's `delete_contact` tool handles requests like "delete the duplicate entry for Acme Corp." The database contains similarly named records (e.g., "Acme Corp," "Acme Corporation," "ACME Corp Inc."), and analytics show 8% of deletions are reversed within 24 hours due to misidentified records. Users have also complained that the current multi-step confirmation flow adds too much friction to routine cleanup tasks. Which approach most effectively reduces the error rate while maintaining workflow efficiency?

- (A) Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion.
- (B) Require users to supply the exact record ID from the CRM interface rather than using natural language references to contact names.
- (C) Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests.
- (D) Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow.

Answer: (A) Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion.

EXPLANATION

Correct. This directly addresses ambiguity by showing clear distinctions between similar records while keeping the workflow fast with a lightweight confirmation step.

Why the other options are wrong

- **B.** Incorrect. This reduces errors but adds significant friction and hurts usability for routine tasks.
- **C.** Incorrect. Helpful as a separate improvement, but it doesn't solve incorrect deletions during manual requests.
- **D.** Incorrect. This mitigates impact after errors occur but does not reduce the error rate itself.

147 After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats - inline citations vs. bibliographies, methodology sections vs. details embedded in introductions. What's the most effective way to address these failures?

- (A) Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.
- (B) Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract.
- (C) Add few-shot examples demonstrating extractions from documents with varied structures - showing how to identify citations in different formats and locate methodology details across section types.
- (D) Implement retry logic that re-sends requests when validation detects empty required fields.

Answer: (C) Add few-shot examples demonstrating extractions from documents with varied structures - showing how to identify citations in different formats and locate methodology details across section types.

EXPLANATION

Correct. This directly improves the model's ability to generalize across diverse document formats, addressing the root cause of missed extractions.

Why the other options are wrong

- **A.** Incorrect. This lowers data quality standards and avoids solving the extraction problem.
- **B.** Incorrect. Regex approaches are brittle and unreliable across varied formats, especially for complex structures like methodology.
- **D.** Incorrect. Retries without improving guidance will likely produce the same incomplete outputs.

148 The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, negative). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than indicating the information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was.. interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. Which modification best addresses both issues?

- (A) Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum.
- (B) Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.
- (C) Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.
- (D) Allow null values for pros/cons, and add "unclear" to the sentiment enum.

Answer: (B) Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.

EXPLANATION

Correct. This prevents fabrication by allowing explicitly empty outputs when no details are present, and "unclear" handles ambiguous or sarcastic sentiment appropriately.

Why the other options are wrong

- **A.** Incorrect. Making fields optional can lead to inconsistent outputs, and "neutral" doesn't solve ambiguity - it's different from "unclear".
- **C.** Incorrect. This adds complexity but doesn't prevent fabrication or resolve ambiguity in outputs.
- **D.** Incorrect. Nulls are less consistent than empty arrays for structured outputs and can complicate downstream processing.

- 149.** Your extraction system implements automatic retries when validation fails. On each retry, the specific validation error is appended to the prompt. This retry-with-error-feedback approach resolves most failures within 2-3 attempts. For which failure pattern would additional retries be LEAST effective?
- (A) The model extracts "et al." for co-authors when the full list exists only in an external document not in the input.
 - (B) The model extracts citation counts as locale-formatted strings ("1234") when the schema requires integers.
 - (C) The model extracts dates as ISO 8601 datetime strings ("2003-03-15T00:00:00Z") when the schema requires only the date portion.
 - (D) The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings.

Answer: (A) The model extracts "et al." for co-authors when the full list exists only in an external document not in the input.

EXPLANATION

Correct. Retries won't help because the required information is not present in the input context. The model cannot recover missing data through repeated attempts.

Why the other options are wrong

- **B.** Incorrect. This is a formatting issue that can be corrected through retries with validation feedback.
- **C.** Incorrect. Also a format mismatch, which retries can fix easily.
- **D.** Incorrect. This is a structural mismatch that can typically be corrected with retry feedback.

- 150.** Your invoice extraction uses tool use with strict JSON schemas. JSON syntax errors never occur, but 12% of extractions fail semantic validation - for example, line item amounts don't sum to the extracted total, or vendor IDs don't match valid formats. These failures currently route to manual review. What's the most effective approach to reduce manual review volume while maintaining accuracy?

- (A) Retry the extraction up to 3 times when validation fails, accepting the first result that passes validation.
- (B) Implement post-processing logic that automatically corrects common errors, such as recalculating totals from line items when sums don't match.
- (C) When validation fails, make a follow-up request with the document, extraction, and validation errors for model correction.
- (D) Add stricter schema constraints with detailed field descriptions to prevent the model from generating invalid values initially.

Answer: (C) When validation fails, make a follow-up request with the document, extraction, and validation errors for model correction.

EXPLANATION

Correct. This provides targeted feedback, enabling the model to fix specific issues, significantly reducing manual review while maintaining accuracy.

Why the other options are wrong

- **A.** Incorrect. Retries without targeted feedback often repeat the same mistakes and don't reliably fix semantic inconsistencies.
- **B.** Incorrect. While useful for specific cases, this is narrow and brittle, and doesn't address broader validation failures like incorrect IDs.
- **D.** Incorrect. Schema improvements help upfront, but they cannot fully prevent semantic errors like mismatched totals.

- 151.** Your team is extracting structured data from 50,000 legacy legal contracts under a two-week deadline. Initial testing with 500 sample documents shows 82% pass JSON schema validation on the first attempt, while the remaining 18% fail due to diverse issues - missing required fields, malformed dates, and incorrectly identified parties. Documents that fail typically need refinements targeting their specific failure modes before extraction succeeds. Which batch processing strategy is the most cost-efficient while still meeting the deadline?

- (A) Split documents into 10 sequential batches of 5,000 each, analysing results and refining prompts between batches to improve extraction quality progressively.
- (B) Submit all 50,000 documents via batch API, then submit failed extractions in successive batches - refining prompts between each batch - until all documents pass validation.
- (C) Use the real-time API for all 50,000 documents since the batch API's 24-hour processing window creates unacceptable deadline risk.
- (D) Process 2,000 sample documents via real-time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts.

Answer: (B) Submit all 50,000 documents via batch API, then submit failed extractions in successive batches - refining prompts between each batch - until all documents pass validation.

EXPLANATION

Correct. This maximizes throughput and parallelism upfront, ensuring the deadline is met. Then it uses targeted iterative refinement only on failures, making it cost-efficient while handling diverse failure modes effectively.

Why the other options are wrong

- **A.** Incorrect. This introduces unnecessary sequential delays and reduces throughput, risking the deadline.
- **C.** Incorrect. This is unnecessarily expensive and not required given batch processing capabilities.
- **D.** Incorrect. While proactive, this assumes failure patterns generalize well, which the scenario suggests they don't - since failures are diverse and require case-specific refinements.

152. Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies. What's the most effective approach to improve extraction accuracy for documents with amendments?

- (A) Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step.
- (B) Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review.
- (C) Redesign the schema so amended fields capture multiple values, each with source location and effective date.
- (D) Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms.

Answer: (C) Redesign the schema so amended fields capture multiple values, each with source location and effective date.

EXPLANATION

Correct. This preserves both original and amended values with context, enabling accurate interpretation and avoiding ambiguity about which value applies.

Why the other options are wrong

- **A.** Incorrect. This is brittle and risky - accurately identifying and removing superseded clauses is complex and may lead to loss of important context.
- **B.** Incorrect. This increases manual review but does not improve extraction accuracy or resolve ambiguity.
- **D.** Incorrect. This relies on model judgment, which is inconsistent, and loses traceability of how values changed over time.

153. Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON. What approach provides the most reliable schema compliance?

- (A) Define a tool with your target schema as input parameters and have Claude call it with the extracted data.
- (B) Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.
- (C) Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails.
- (D) Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.

Answer: (A) Define a tool with your target schema as input parameters and have Claude call it with the extracted data.

EXPLANATION

Correct. Tool use enforces strict schema compliance at generation time, ensuring valid, structured JSON that downstream systems can reliably consume.

Why the other options are wrong

- **B.** Incorrect. This is a fragile workaround and does not guarantee valid or schema-compliant JSON.
- **C.** Incorrect. Helpful but not reliable - models can still produce malformed or non-conformant JSON.
- **D.** Incorrect. Prompt-based formatting alone cannot guarantee strict compliance, especially in edge cases.

154. Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract skills mentioned." What's the most effective improvement?

- (A) Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."
- (B) Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.
- (C) Enrich the schema to capture extraction metadata.
- (D) Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

Answer: (D) Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

EXPLANATION

Correct. Examples directly guide the model on how to split, what to include, and the expected level of detail, addressing all three issues effectively.

Why the other options are wrong

- **A.** Incorrect. This enforces limits but is arbitrary and may exclude valid skills or still leave ambiguity in how to split phrases.
- **B.** Incorrect. Helpful downstream, but it does not fix inconsistent extraction behavior at the source.
- **C.** Incorrect. Adds complexity but does not directly address inconsistency in skill identification and formatting.

155. Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- (A) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- (B) Set `tool_choice` to "any" so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.
- (C) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.
- (D) Set `tool_choice` to "auto" and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.

Answer: (A) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.

EXPLANATION

Correct. This enforces the correct execution order, ensuring required data (like DOI) is available before dependent tools are called.

Why the other options are wrong

- **B.** Incorrect. This does not guarantee ordering - Claude may still choose the wrong tool first.
- **C.** Incorrect. This is too rigid and prevents legitimate use of other tools in later steps.
- **D.** Incorrect. Tool ordering does not reliably control selection or execution order.

156. Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence >90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

- (A) Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.
- (B) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- (C) Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.
- (D) Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.

Answer: (B) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

EXPLANATION

Correct. Aggregate accuracy can hide weak spots. You need to ensure confidence >90% is trustworthy across all segments, otherwise automation may introduce systematic errors.

Why the other options are wrong

- **A.** Incorrect. Important, but it doesn't ensure the confidence signal is reliable across different cases - it only checks overall acceptability.
- **C.** Incorrect. Useful for tuning, but only after confirming the confidence signal is consistent and reliable across segments.
- **D.** Incorrect. A pilot is valuable, but deploying without validating segment-level reliability first introduces avoidable risk.

157. Your extraction system uses `tool_use` with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model's context window is 200K tokens. What is the most likely cause?

- (A) Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.
- (B) Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions.
- (C) The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus.
- (D) Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length.

Answer: (A) Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.

EXPLANATION

Correct. The tool schema (~2,500 tokens) plus system prompts and large documents push total input close to the 200K context limit, causing truncation or reduced attention to the final portion - hence missed information in the last third.

Why the other options are wrong

- **B.** Incorrect. While attention can vary, the sharp drop near the context boundary strongly indicates a context limit issue, not general attention decay.
- **C.** Incorrect. This is a weaker effect and does not explain the consistent failure in the final third tied to document size thresholds.
- **D.** Incorrect. Schema size is constant across cases; it does not explain why accuracy drops only for longer documents.

158. Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total - sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals. What's the most effective approach to improve extraction reliability?

- (A) Add a "calculated_total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ.
- (B) Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct.
- (C) Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.
- (D) Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.

Answer: (A) Add a "calculated_total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ.

EXPLANATION

Correct. This preserves both sources of truth and enables reliable validation. Discrepancies can be flagged explicitly, improving accuracy without silently altering financial data.

Why the other options are wrong

- **B.** Incorrect. This adds complexity and uncertainty - "guessing" which value is correct can introduce errors in financial data.
- **C.** Incorrect. Helpful but insufficient - does not handle OCR errors or real inconsistencies in source documents.
- **D.** Incorrect. This modifies financial data artificially, which is risky and unacceptable for accounting accuracy.

159. Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- (A) Submit all documents to the Batch API with custom_ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.
- (B) Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- (C) Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- (D) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

Answer: (D) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

EXPLANATION

Correct. This balances cost efficiency and latency requirements, ensuring urgent reports are processed quickly while optimizing cost for non-urgent ones.

Why the other options are wrong

- **A.** Incorrect. Batch processing introduces delays that can exceed the 30-minute requirement, making it unsuitable for urgent reports.
- **B.** Incorrect. This meets latency needs but is unnecessarily expensive for standard reports that don't require real-time processing.
- **C.** Incorrect. Hourly batching further increases delay and cannot meet urgent processing requirements.

160The extraction pipeline receives documents of varying types - some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own schema tailored to the document type. During testing, you observe that with `tool_choice: "auto"`, Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

- (A) Consolidate all document types into a single unified-schema extraction tool and force that tool.
- (B) Keep `tool_choice: "auto"` with system prompt instructions requiring tool use.
- (C) Set `tool_choice: "any"` with all extraction tools defined.
- (D) Add a preliminary classification call, then make a second call with `tool_choice` forced to the identified extraction tool.

Answer: (C) Set `tool_choice: "any"` with all extraction tools defined.

EXPLANATION

Correct. This guarantees that a tool will always be called, eliminating conversational text responses. While tool selection may not always be perfect, it ensures structured output every time, which is the primary requirement.

Why the other options are wrong

- **A.** Incorrect. This reduces extraction accuracy and creates an overly complex schema that doesn't fit all document types well.
- **B.** Incorrect. Instructions alone are not enforceable, which is exactly why the issue occurs.
- **D.** Incorrect. While more precise, this adds extra latency and complexity. The question prioritizes guaranteed structured output, which C achieves more directly.

161Monitoring shows 12% of extractions fail Pydantic validation with specific errors like "expected float for quantity, got '2 to 3'". Retrying these requests without modification produces the same failures. What's the most effective approach to recover from these validation failures?

- (A) Set temperature to 0 to eliminate output variability and ensure consistent formatting.
- (B) Send a follow-up request including the validation error, asking the model to correct its output.
- (C) Pre-process source documents to standardize problematic formats before sending them for extraction.
- (D) Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation.

Answer: (B) Send a follow-up request including the validation error, asking the model to correct its output.

EXPLANATION

Correct. Providing specific validation feedback allows the model to correct the exact issue (e.g., convert "2 to 3" into a valid float), making recovery highly effective.

Why the other options are wrong

- **A.** Incorrect. This reduces randomness but won't fix systematic extraction errors like misinterpreting ranges ("2 to 3").
- **C.** Incorrect. Helpful in some cases, but not scalable or sufficient for diverse real-world variations.
- **D.** Incorrect. More expensive and not necessary - targeted correction is more efficient and effective.

162After three months of weekly sessions, your conversation history has grown to 85,000 tokens. When users ask "What did we conclude about the theme of isolation?", the assistant provides generic literary analysis rather than referencing the group's specific insights from earlier sessions. Discussions often build on previous meetings' conclusions, so maintaining narrative context is important. What's the most effective approach?

- (A) Add structured XML tags to mark significant discussion conclusions throughout the conversation history.
- (B) Use semantic embedding to index the full conversation history and retrieve only relevant past exchanges for each user query, replacing the linear conversation format with retrieved segments.
- (C) Implement rolling window truncation to keep only the most recent 25,000 tokens.
- (D) Implement progressive summarization where older conversation blocks are replaced with concise summaries that explicitly extract key conclusions, decisions, and recurring themes, keeping recent exchanges verbatim.

Answer: (D) Implement progressive summarization where older conversation blocks are replaced with concise summaries that explicitly extract key conclusions, decisions, and recurring themes, keeping recent exchanges verbatim.

EXPLANATION

Correct. This preserves long-term context and key insights while staying within token limits, enabling the assistant to reference past conclusions effectively.

Why the other options are wrong

- **A.** Incorrect. Tagging helps organization, but it doesn't solve the context length limitation or ensure relevant retrieval at query time.
- **B.** Incorrect. Powerful for retrieval, but replacing the conversation entirely loses narrative continuity, which is important for ongoing discussions.
- **C.** Incorrect. This discards earlier insights, which are critical for referencing past conclusions.

163After a 40-minute session helping plan a dinner party, the conversation has grown to 78,000 tokens. The history includes: (1) the user mentioning a guest has a severe shellfish allergy, (2) measurements for scaling recipes to 8 servings, (3) the user's clarification that "room temperature butter" means 68F in their kitchen, and (4) general back-and-forth about meal timing and presentation. You need to implement context management before the window limit is reached. What approach best balances information preservation with token reduction?

- (A) Summarize the entire conversation history into a concise summary capturing main topics discussed, then append new messages going forward.
- (B) Implement a sliding window retaining only the most recent 20,000 tokens, relying on users to re-state important information when relevant.
- (C) Store the full conversation externally and use semantic search to retrieve relevant portions for each turn, loading only matching segments into context.
- (D) Extract critical structured data (allergies, serving counts, user-defined terms) into a compact reference section, summarize general discussion, and retain recent exchanges verbatim.

Answer: (D) Extract critical structured data (allergies, serving counts, user-defined terms) into a compact reference section, summarize general discussion, and retain recent exchanges verbatim.

EXPLANATION

Correct. This preserves high-priority facts exactly, reduces tokens efficiently, and maintains conversational continuity.

Why the other options are wrong

- **A.** Incorrect. This risks losing critical details like allergies and precise measurements that must remain exact.
- **B.** Incorrect. This discards essential context (e.g., allergy info), which is unsafe and unreliable.
- **C.** Incorrect. Helpful for retrieval, but may miss important persistent constraints (like allergies) if not explicitly retrieved every time.

164.You're implementing a feature where users refine their playlist preferences through multiple conversation turns. After deploying, you notice Claude's responses don't reflect what users said earlier in the same conversation - for example, a user says they love jazz, but two messages later Claude asks what genres they enjoy. What is the most likely cause?

- (A) The model's context window has been exceeded by the conversation length.
- (B) The Claude API requires a session_id parameter that you haven't configured.
- (C) Claude requires a vector database connection to maintain conversation memory.
- (D) Your application isn't including prior messages in the messages array.

Answer: (D) Your application isn't including prior messages in the messages array.

EXPLANATION

Correct. Claude does not retain memory between calls - if previous messages aren't included, it cannot recall earlier user preferences.

Why the other options are wrong

- **A.** Incorrect. This would only happen in very long conversations, and the issue appears early within just a few turns.
- **B.** Incorrect. There is no required session_id - context must be explicitly managed by the application.
- **C.** Incorrect. A vector database is optional for retrieval, not required for basic conversation memory.

165.Your fitness coaching assistant uses a system prompt with detailed conditional logic: "If the user mentions being a beginner, provide step-by-step form instructions. If they use terms like 'progressive overload' or 'superset', respond concisely. If they ask about injury history, always recommend consulting a physician." During evaluation, you find the assistant handles explicit expertise declarations correctly but struggles when users don't clearly state their level - often defaulting to overly detailed responses regardless of contextual cues like technical terminology. Which change to the system prompt would most directly address this failure to pick up on implicit expertise signals?

- (A) Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about physician consultations.
- (B) Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced."
- (C) Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses.
- (D) Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message.

Answer: (A) Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about physician consultations.

EXPLANATION

Correct. This shifts from brittle rule-based logic to a flexible heuristic, enabling the model to infer expertise from subtle cues like terminology and tone.

Why the other options are wrong

- **B.** Incorrect. This increases complexity and brittleness - edge cases will still be missed.
- **C.** Incorrect. Useful, but doesn't solve the issue of interpreting implicit signals during conversation.
- **D.** Incorrect. This adds friction and interrupts flow instead of improving implicit understanding.

166 During initial testing, you notice that Claude doesn't seem to remember vocabulary words from earlier in the conversation. When a student asks "Can you quiz me on those words?", Claude responds as if no words have been discussed. What is the most likely explanation?

- (A) Your system prompt needs explicit instructions telling Claude to remember information from earlier turns.
- (B) You're not including prior messages in each API request - the stateless API doesn't retain conversation history.
- (C) You need to enable conversation persistence by passing a session ID parameter with each API call.
- (D) The model's context window has filled up, causing earlier conversation content to be dropped.

Answer: (B) You're not including prior messages in each API request - the stateless API doesn't retain conversation history.

EXPLANATION

Correct. Claude is stateless. If earlier messages aren't passed in the request, it has no awareness of prior vocabulary.

Why the other options are wrong

- **A.** Incorrect. Instructions alone don't give the model memory - context must be provided with each request.
- **C.** Incorrect. There's no required session ID - memory is handled by including past messages.
- **D.** Incorrect. This would only happen in very long conversations, not typical early testing scenarios.

167 Your home renovation planning assistant uses a system prompt defining an expert contractor persona with specific guidelines: always ask about budget, suggest alternatives at multiple price points, and confirm timeline requirements. During testing, responses follow these guidelines for turns 1-4, but by turn 7, the assistant gives generic advice without asking about budget or timeline. The conversation totals only 2,500 tokens. What is the most likely cause?

- (A) System prompts only establish initial behavior and don't persist across all turns.
- (B) The system prompt is only sent with the first API request.
- (C) The assistant's accumulated responses are diluting the system prompt's influence.
- (D) The model's attention on system prompt instructions naturally weakens as turns accumulate.

Answer: (C) The assistant's accumulated responses are diluting the system prompt's influence.

EXPLANATION

Correct. As the conversation grows, the model must balance more context. Even within a small token count, recent conversational patterns can outweigh earlier instructions, causing drift from the original system prompt.

Why the other options are wrong

- **A.** Incorrect. System prompts do persist as long as they are included in each request.
- **B.** Incorrect. If this were the case, the behavior would degrade immediately after the first turn - not gradually by turn 7.
- **D.** Incorrect. This is too vague - it's not just "natural weakening," but specifically the increasing influence of conversational context relative to the system prompt.

168 Users report that during extended conversations, the AI loses track of specific topics, examples, and preferences they mentioned earlier in the session. Your current implementation uses a sliding window that keeps only the most recent 25 message pairs to stay within context limits. What's the most effective approach to maintain awareness of earlier conversation content while managing context size?

- (A) Replace the sliding window with a hybrid approach: summarize older messages while keeping recent messages verbatim.
- (B) Implement vector similarity search over the full conversation history, retrieving relevant past messages for each user query.
- (C) Increase the window size to 50 message pairs to retain more conversation history before truncation.
- (D) Add a separate API call each turn to summarize messages being dropped, prepending this running summary to the conversation.

Answer: (A) Replace the sliding window with a hybrid approach: summarize older messages while keeping recent messages verbatim.

EXPLANATION

Correct. This preserves long-term context in a compressed form while keeping recent interactions intact, maintaining continuity without exceeding token limits.

Why the other options are wrong

- **B.** Incorrect. While powerful, this is more complex and better suited for large-scale or cross-session retrieval, not necessarily the most direct fix for in-session context loss.
- **C.** Incorrect. This only delays the issue and increases token usage without fundamentally solving it.
- **D.** Incorrect. This can lead to summary drift and compounding errors, reducing reliability over time.

169 Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4 questions before taking any action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't match preferences. What's the most effective approach to improve this trade-off?

(A) Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendation.

(B) Configure the assistant to proceed with reasonable defaults (medium sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to correct them if results don't match expectations.

(C) Instruct the assistant to state explicit assumptions based on conversation status, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions like confirming bookings.

(D) Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total.

Answer: (C) Instruct the assistant to state explicit assumptions based on conversation status, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions like confirming bookings.

EXPLANATION

Correct. This balances speed and accuracy - users get immediate value, assumptions are transparent, and friction is minimized.

Why the other options are wrong

- **A.** Incorrect. This ensures completeness but adds high friction, increasing drop-off before any value is delivered.
- **B.** Incorrect. Hidden assumptions can lead to misalignment and confusion, reducing trust.
- **D.** Incorrect. This reduces question count but still creates front-loaded friction without delivering immediate value.

170 During QA testing, you notice that Claude follows your system prompt guidelines consistently in the first 10-15 turns, but by turn 25-30, responses begin deviating - using informal tone when formality was specified, occasionally skipping required formatting, or providing information types the guidelines restrict. Conversation length is well within context limits (typically 30,000 tokens out of 200,000 available). What's the most effective approach to maintain consistent behavior throughout extended conversations?

(A) Insert user-role messages that reinforce critical guidelines at natural conversation breakpoints, especially before complex requests.

(B) Implement post-response validation that regenerates each response until it conforms to the specified guidelines.

(C) Automatically start a new conversation after 20 turns, passing a summary of the prior context to maintain continuity.

(D) Move behavioral guidelines from the system prompt into the first user message.

Answer: (A) Insert user-role messages that reinforce critical guidelines at natural conversation breakpoints, especially before complex requests.

EXPLANATION

Correct. This combats instruction drift by periodically refreshing key constraints within the active context, keeping behavior aligned over long conversations.

Why the other options are wrong

- **B.** Incorrect. This is costly and reactive, and can create loops without addressing the root cause of drift.
- **C.** Incorrect. Summarization can lose detail and doesn't directly reinforce behavioral rules.
- **D.** Incorrect. User messages carry less weight than system prompts and are more likely to be overridden during the conversation.

171 Performance analysis reveals your context is composed of accumulated RAG results from all previous queries, which is crowding out conversation history and causing coherence degradation after 15+ turns. Which approach best addresses this issue?

(A) Implement semantic deduplication to identify and remove redundant information across the accumulated RAG results and conversation turns.

(B) Implement a sliding window for RAG results from the last 2-3 queries while preserving conversation history.

(C) Shift context budget to favor RAG results while reducing conversation history allocation.

(D) Compress all RAG results into a consolidated summary document that updates incrementally after each retrieval.

Answer: (B) Implement a sliding window for RAG results from the last 2-3 queries while preserving conversation history.

EXPLANATION

Correct. This directly addresses context crowding by limiting RAG growth while keeping conversation continuity intact.

Why the other options are wrong

- **A.** Incorrect. This reduces redundancy but doesn't solve the core issue of unbounded accumulation of RAG context.
- **C.** Incorrect. This worsens coherence by sacrificing conversation context.
- **D.** Incorrect. This can introduce information loss and summary drift, especially across many turns.

